

综合系统开发

```
<%@ page contentType="text/html"%>
<%@page pageEncoding="GB2312"%>
<%@page import="java.sql.*" %> <!--导入java.sql包-->
<html>
<head>
<title>从MySQL数据库中读出student表</title>
</head>
<body bgcolor="orange">
<%
```

```
    try {
        Class.forName("com.mysql.jdbc.Driver");
        String url = "jdbc:mysql://localhost:3306/test"; //数据库名
        String username = "root"; //数据库用户名
        String password = "root"; //数据库用户密码
        Connection conn = DriverManager.getConnection(url, username, password);

        if(conn != null){
            out.print("数据库连接成功! ");
            out.print("<br />");
        }
    }
%>
<table border="2">
<tr>
<th width="87" align="center">学号</th>
    <th width="87" align="center">姓名</th>
    <th width="87" align="center">性别</th>
    <th width="87" align="center">年龄</th>
    <th width="87" align="center">体重</th>
</tr>
```

jsp

Html、java代码混合,
不易修改

```
Statement stmt = null;
ResultSet rs = null;
String sql = "SELECT * FROM student;"; //查询语句
stmt = conn.createStatement();
rs = stmt.executeQuery(sql);
out.print("查询结果: ");
out.print("<br/>");
while (rs.next()) {%>
```

```
<tr>
    `在这里插入代码片`<td><%=rs.getString("SID")%></td>>
    <td><%=rs.getString("SName")%></td>>
    <td><%=rs.getString("SSex")%></td>>
    <td><%=rs.getString("SAge")%></td>>
    <td><%=rs.getString("SWeight")%></td>>
```

```
</tr>
<%
```

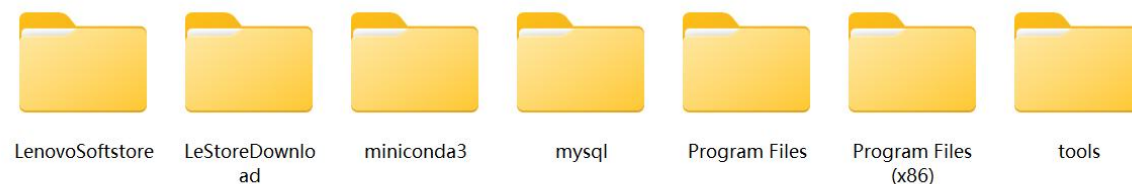
```
    }
    }else{
        out.print("连接失败! ");
    }
}catch (Exception e) {
    //e.printStackTrace();
    out.print("数据库连接异常! ");
}
```

```
%>
</table>
</body>
</html>
```

1 MVC

- 全名Model View Controller，最早由 Trygve Reenskaug 在 1978 年提出，是一种**软件设计模式**。开始用在桌面程序中，后来[Sun](#)公司[Java EE](#)。用**业务逻辑、数据与界面显示分离**的方法来组织代码，在需要改进和个性化定制界面及用户交互的同时，不需要重新编写业务逻辑，达到减少编码的时间。

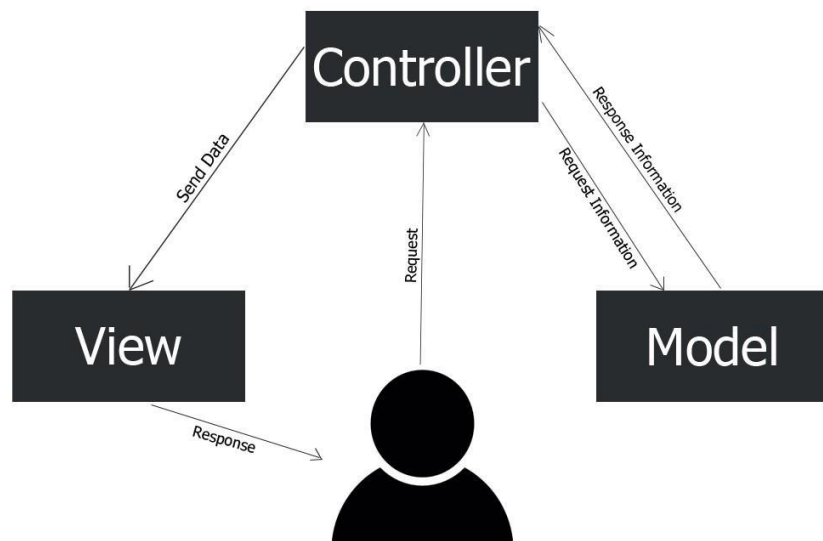
名称	修改日期	类型
LenovoSoftstore	2022/3/25 15:11	文件夹
LeStoreDownload	2022/5/5 8:00	文件夹
miniconda3	2022/5/5 10:29	文件夹
mysql	2022/5/11 15:18	文件夹
Program Files	2022/5/5 10:20	文件夹
Program Files (x86)	2022/5/11 8:57	文件夹
tools	2022/5/12 9:24	文件夹



MVC三个部件是相互独立，改变其一不会影响其它两个，构造良好低**耦合**构件。

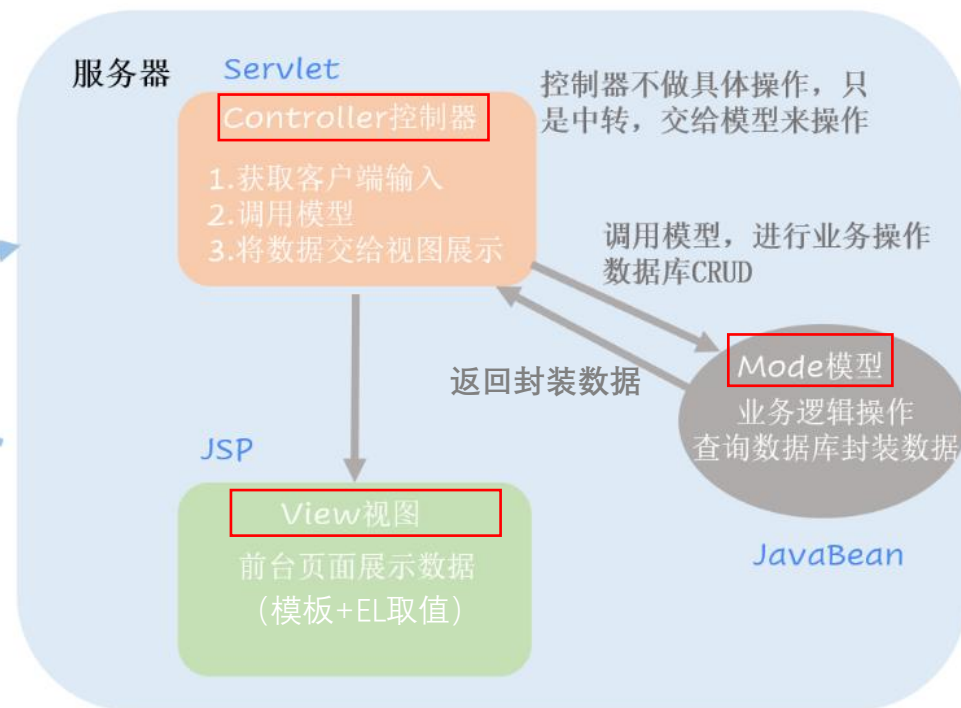
1.1 MVC框架模型

Model-View-Controller



用户-MVC三者关系

浏览器

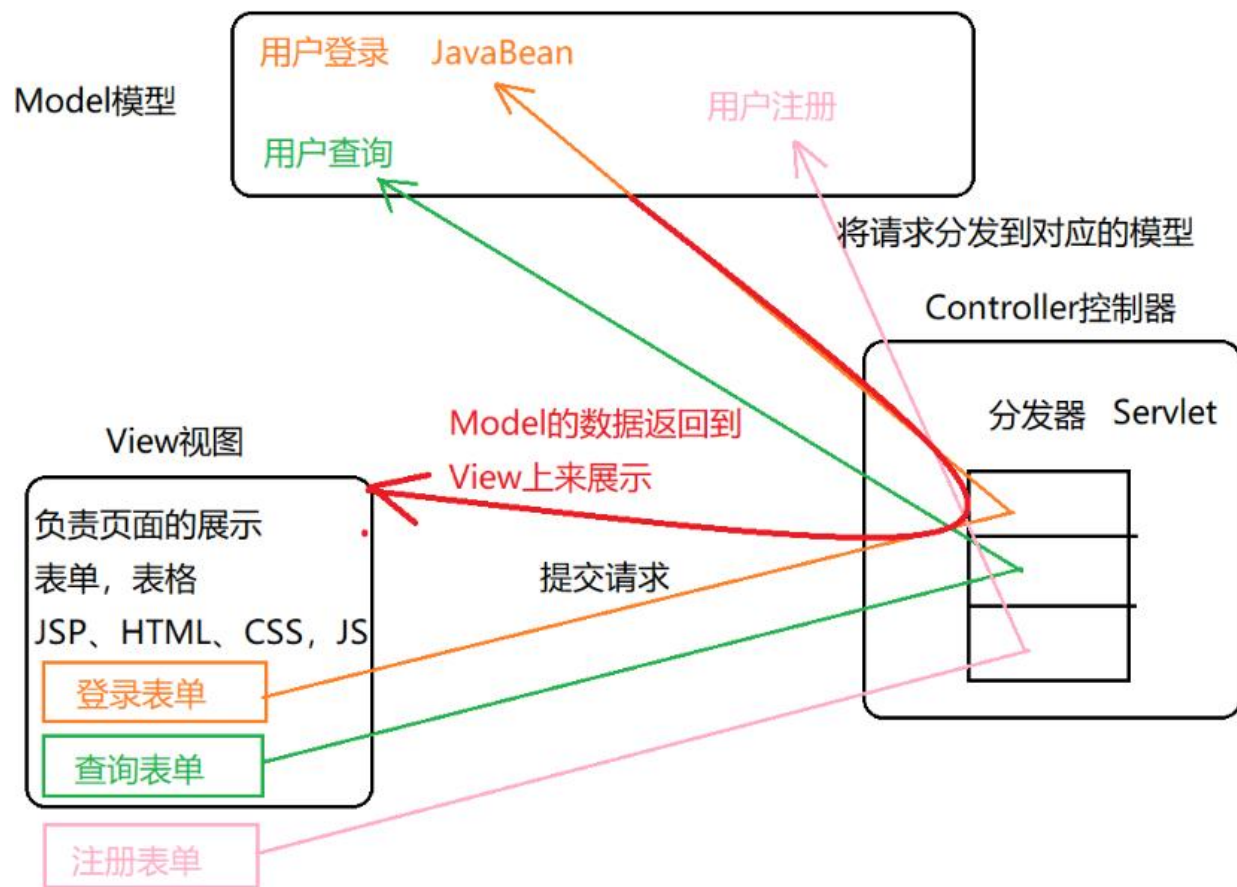


MVC功能

1.2 MVC工作流程--以用户登录进首页index.jsp为例

View层index.jsp, Controller是/loginServlet, Model是JavaBean对象

- 用户在登录页面, 输入数据点击登录按钮, 将请求转发到/loginServlet控制器上
- 然后控制器再将请求分发到Model上, 通过JDBC连接到数据库来查询数据库中是否存在该用户信息。
- 若存在该用户, 则返回信息, 让控制器告诉前台页面展示登陆成功的信息; 不存在则告诉登陆失败...
- View来渲染控制器交给它的数据到index.jsp并在浏览器展示



1.3 优缺点

【优点】

- 1.耦合性低，方便维护，可以利于分工协作
- 2.重用性高

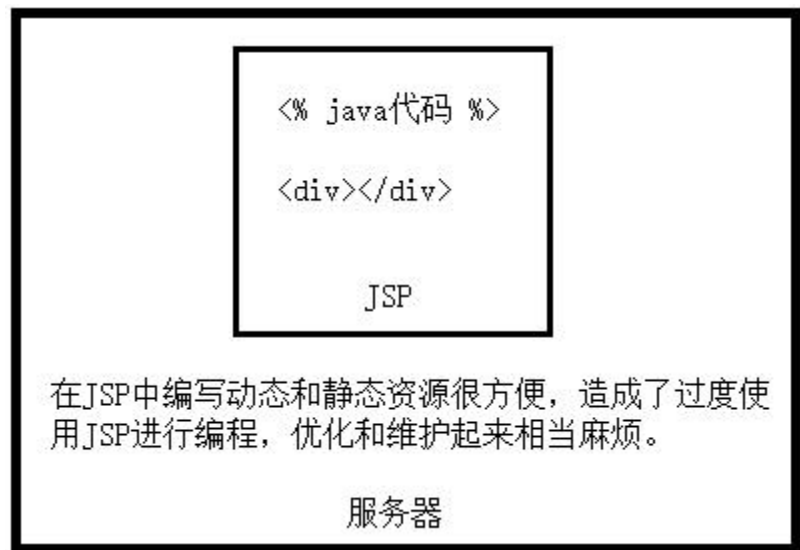
【缺点】

- 1.使得项目架构变得复杂，对开发人员要求高（如所有对`jsp`的访问都要通过`controller`转发，代码层次结构复杂、多）

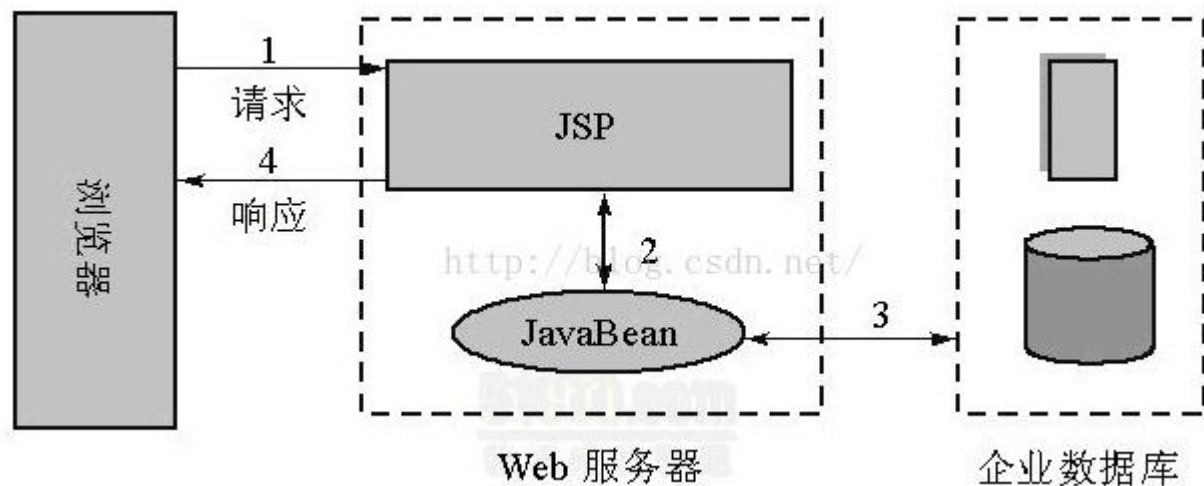
2 JSP开发模型历程

2.1 model1

- “Sun公司为JSP技术提供了两种开发模型: JSP Model1和JSP Model2”
- 传统model1服务器端只有JSP页面，所有的操作都在JSP页面中，包括访问数据库的API也在JSP页面中完成。所有都耦合在一起，对后期的维护和扩展极为不利。



- 优化后model1的开发模式是：jsp+javabean的模式，它的核心是J S P 页面，jsp页面负责整合页面和javabean（业务逻辑），而且渲染页面。基本流程如下：



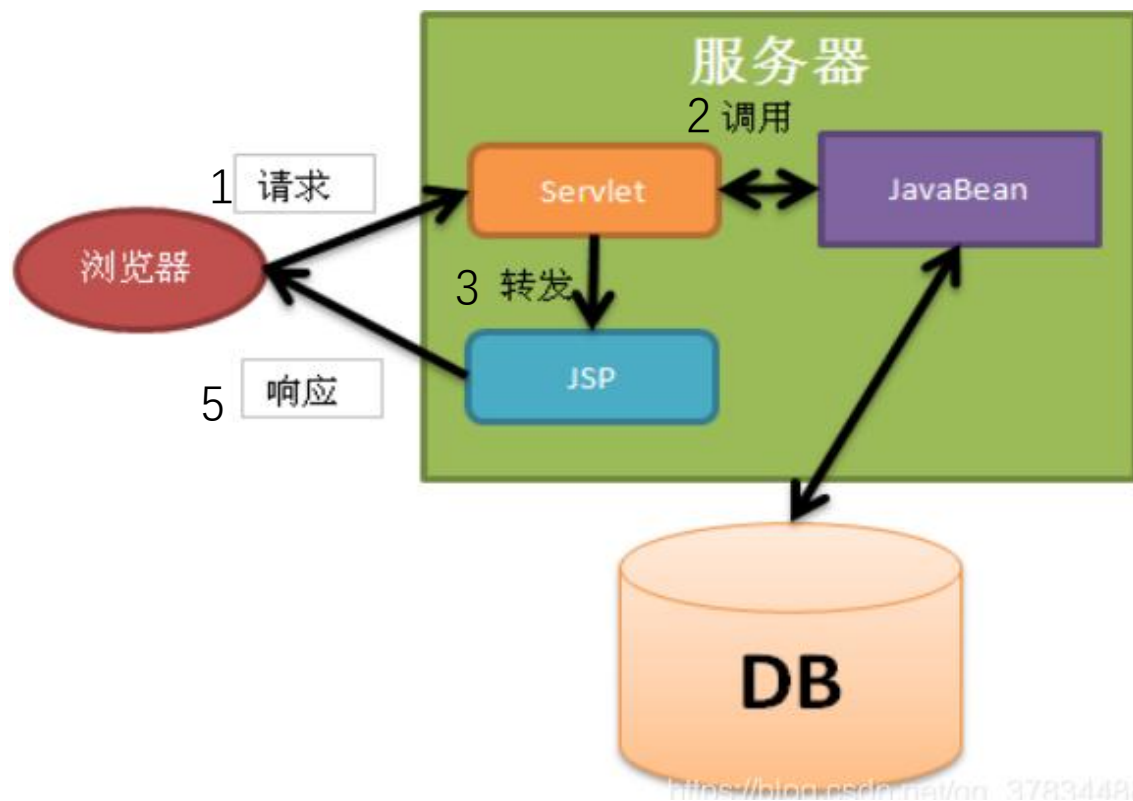
适合小型Web项目，开发成本低！

视图工作和请求调度（控制器）的工作耦合在一起

2.2 model2

model2的开发模式是：Jsp+servlet+javabean的模式，具有MVC完整的结构。

- JSP：视图层，用来与用户打交道。负责接收数据，以及显示数据给用户；
- Servlet：控制层，负责调用合适的模型对象来处理业务逻辑，转发到合适的视图；
- JavaBean：模型层，完成具体的业务工作，例如：开启、转账等。



想让程序发送一个Add Product表单，要发送给控制器 URI: *http://domain/appName/product_input*

为什么model2不直接请求jsp页面，而是通过servlet（如.do文件）来转发 通过tologin.do—>打开login.jsp

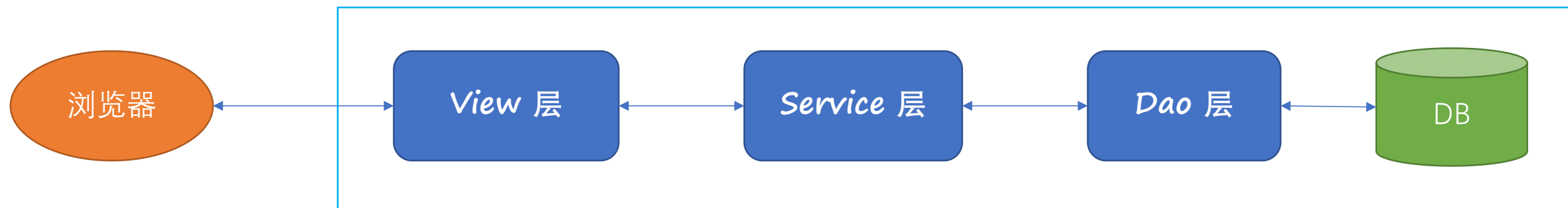
- 1 regist.do对应regist.jsp，通过网页所有的跳转都是跳到regist.do去注册，但直接输入regist.jsp也可以打开去注册。
- 2 便于整体框架的处理。比如，可以加入登陆到某个页面的权限控制，只需要在配置文件中配置，而不用在每个JSP文件中加入相关的逻辑。
- 3 .do一般是servlet，也就是MVC当中的C，是控制器。在整个系统中起到中央枢纽的作用。先通过它的话就可以把逻辑代码隐藏在后台，使JSP文件作为单纯的VIEW分离出来，降低了系统的耦合性。
- 4 访问Controller的路径是以 .do结尾，不方便，springboot可以去掉do后缀，实现路径友好

2.3 三层架构

- MVC模式是**理论基础**，**不是具体实现模式**。实际代码并没有**解耦分离**，特别是**Model**（包括业务、数据访问和实体类、工具类等功能）部分的代码，为了**增强**代码的**维护性**和**降低**代码**耦合性**，将代码分层管理，于是就有了三层架构。三层体系架构指：

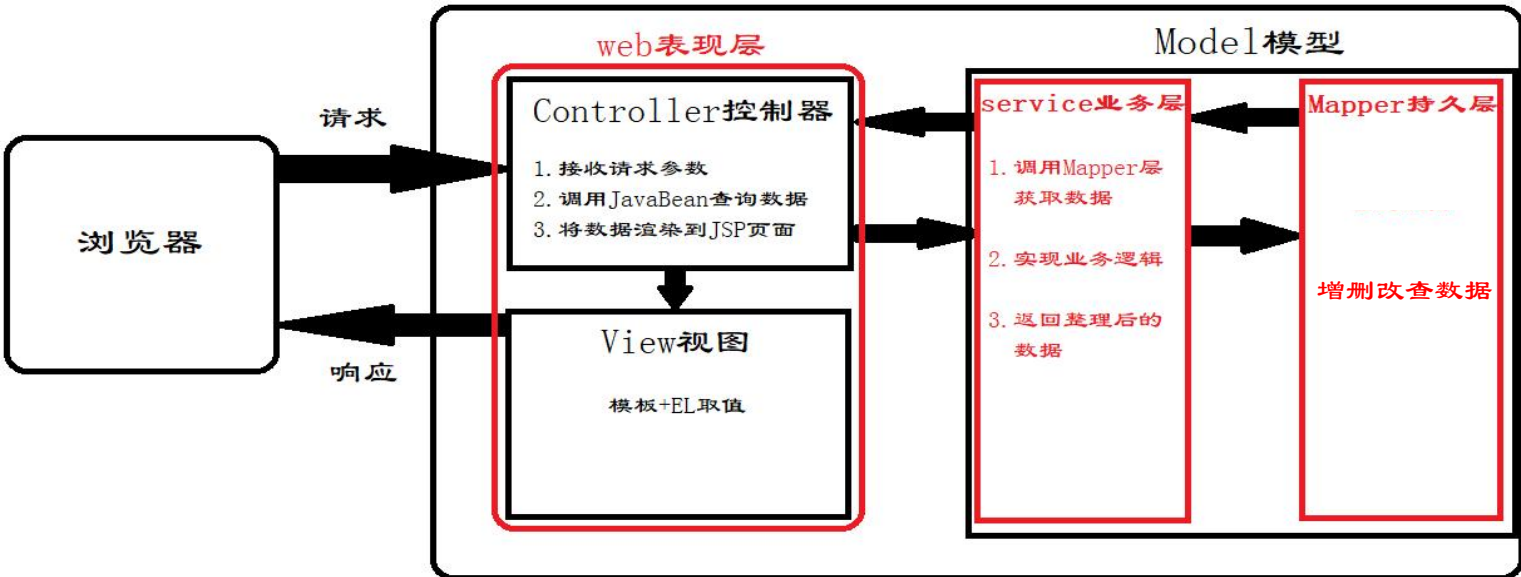
视图层 *View*、服务层 *Service*，与持久层 *Dao*。（没有单独的**controller**）

- *View* (*Web*) 层：用户提交请求或呈现数据。
- *Service* 层：系统的业务逻辑。
- *Dao* (*Mapper*) 层：操作数据库。

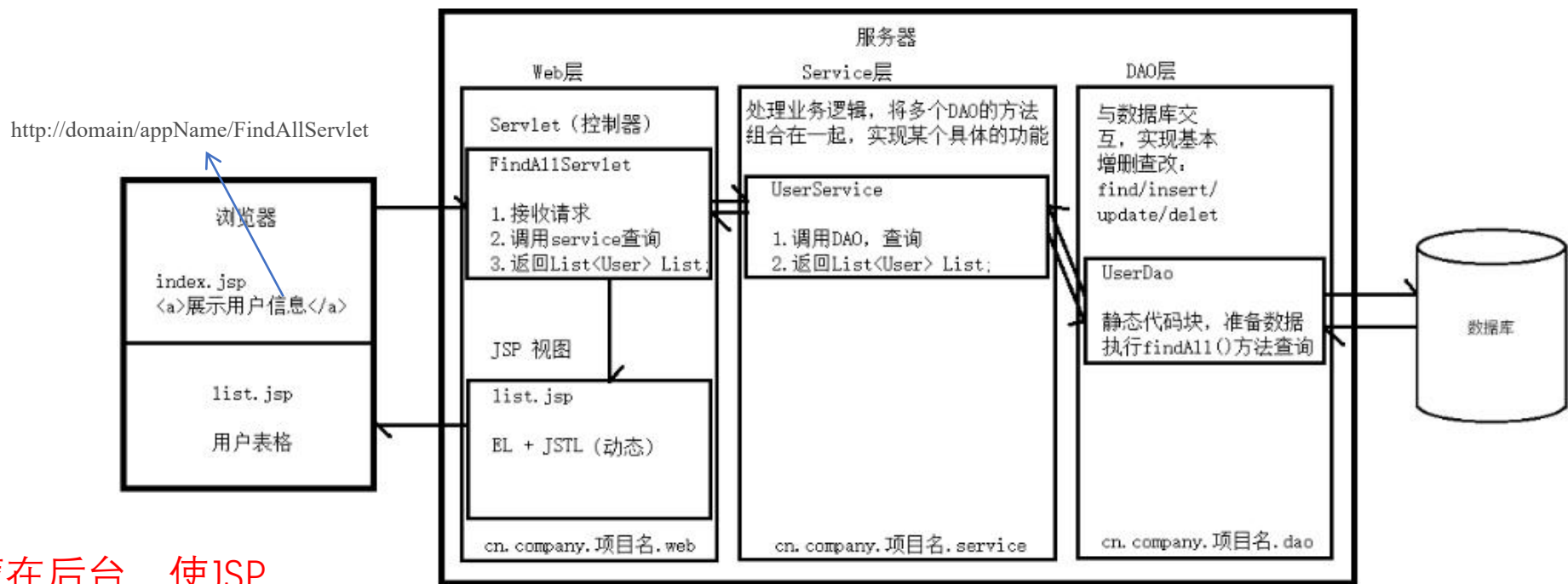


MVC与三层架构的关系

三层中web层对应MVC中的Controller(Servlet)和JSP，其他层都属于MVC中的Model。



示例



各层间会传递一些实体对象:
cn.company.domain (pojo/vo)

先通过servlet把逻辑代码隐藏在后台，使JSP作为单纯的VIEW分离出来，降低了系统的耦合性。

分层的实现

1.显示层： JSP HTML, css ,JS, do处理页面

2.业务逻辑层

services|biz 从数据访问层拿到数据后进行相关判断逻辑处理。分类：

✓ 简单业务逻辑层： 直接从数据访问层拿到数据进行操作。

✓ 复杂业务逻辑层：

从数据访问层拿到数据后， 做筛选

比如：

❑ 删除--从数据访问层拿到对应的数据进行判断， 看删除的数据是否在表中存在。如果存在， 再调用删除的方法删除即可。

❑ 银行转账（oracle事务处理） --当执行转账功能时， 首先要判断是否满足多个条件， 然后执行多项数据库相关写操作， 写操作要么一起成功， 要么一起失败。

3.数据访问层

dao(接口+实现类), utils

从DB中获取数据 增删改查

4.通过实体将三个层进行联系起来。

实体不属于三层

首先在数据库中创建user表，结构如下

id	user_id	order	price
1	1	15275760194821	30
3	2	63882268122909	15
4	3	84634593123730	23
5	4	31026744641728	56
6	5	59192875568090	32
7	3	97122697179471	71
8	5	65662564335452	24
(*)	(Auto)	(NULL)	(NULL)

1 src 目录下创建存放实体类的包：entity，然后在包下创建了user类，它的属性和数据表user的属性一一对应

untitled E:\扬州学习\untitled\untitled

idea

src

main

java

三层架构

BLL

DAL

entity

user

UL

utils

Customer

user

resources

druid.properties

```
1 package 三层架构.entity; /*
2     @author shkstart
3     @create 2023-05-30-10:53
4     */
5
6
7 import java.util.Date;
8
9 public class user {
10     private int id;
11     private String name;
12     private String sex;
```

示例

2 创建数据访问层DAL，在数据访问层中将数据库表进行操作的方法写在接口 UserDao 中

DAL

UserDao

UserDaoImpl

entity

UL

utils

Customer

user

resources

druid.properties

webapp

it

t

xml

ed.iml

控制台

```
6 import 三层架构.entity.user;
7
8 public interface UserDao {
9     //根据id查找学生
10     user findOneById(int id);
11     //验证登录
12     user login(String name, String password);
13     //添加学生
14     user addOne(user user);
15     //修改学生信息
16     user updateMsg(user user);
17     //根据id删除学生
18     void deleteStu(int id);
19 }
```

3 实现类 UserDaoImpl 写具体操作数据库表的逻辑代码

三层架构

BLL

DAL

UserDao

UserDaoImpl

entity

UL

utils

Customer

user

resources

druid.properties

webapp

st

t

xml

ed.iml

```
63
64
65
66
67
68
69
70 String sql = "UPDATE `userinfo` ui,`login_in` li " +
71             "SET ui.`age`=?,ui.`birth`=?,ui.`phone`=?,ui.`" +
72             "WHERE li.`username`=? and ui.`username`=?";
73 String sql1="update userinfo set age =10 where id = 6"
74 update(conn, sql,u.getAge(),u.getBirth(),u.getPhone(),
75         u.getPassword(),u.getName(),u.getName());
```

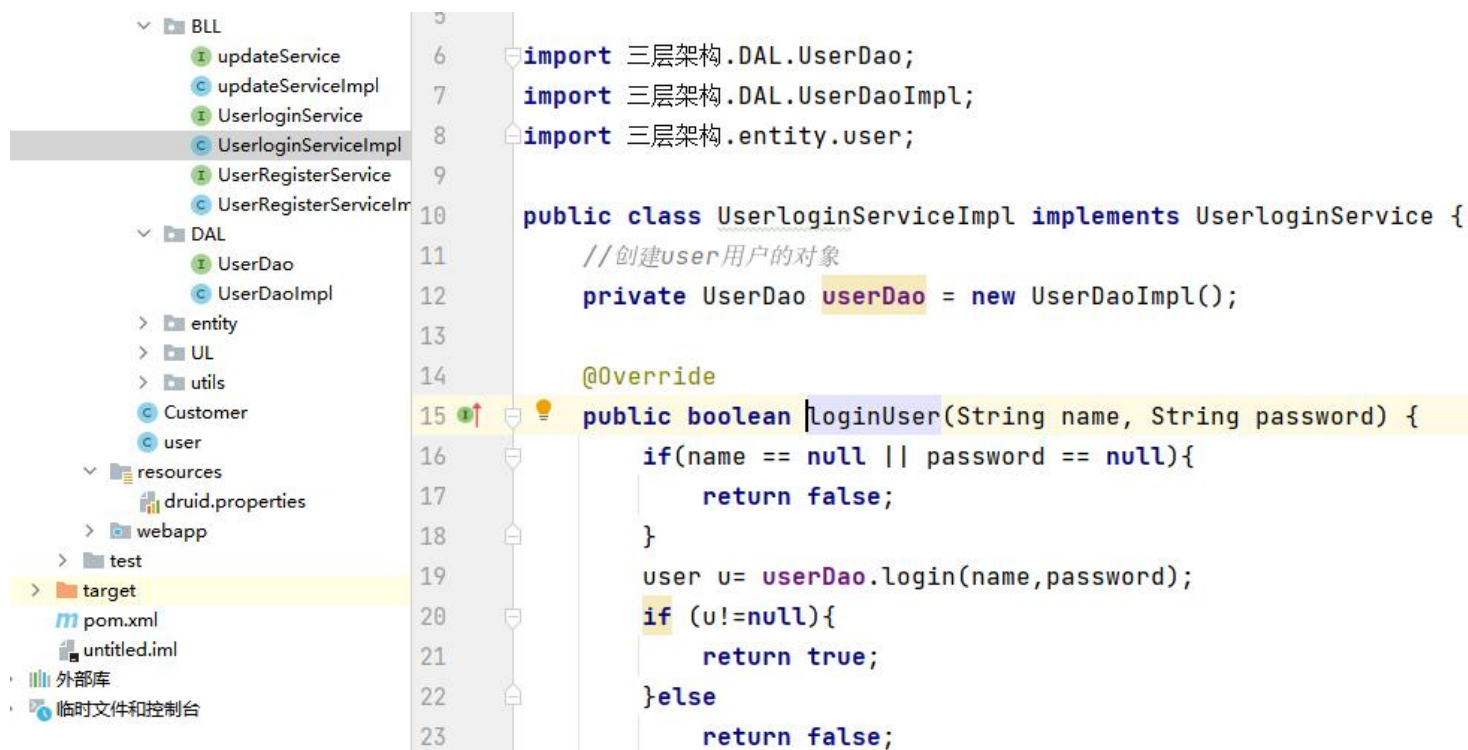
4 创建业务逻辑层接口， 建立接收到表现层 UI 的用户登录指令方法。



The screenshot shows an IDE with four tabs: updateService.java, UserloginServiceImpl.java, UserloginService.java, and logi. The left sidebar shows a project structure with folders '三层架构' and 'BLL', and files 'updateService', 'updateServiceImpl', 'UserloginService', and 'UserloginServiceImpl'. The main editor displays the code for 'UserloginService.java'.

```
1 package 三层架构.BLL; /*
2     @author shkstart
3     @create 2023-06-01-9:50
4 */
5
6 public interface UserloginService {
7     boolean loginUser(String name,String password);
8 }
```

5 业务层接口实现类， 连接数据访问层 DAL进行业务处理



The screenshot shows an IDE with a project structure on the left and the implementation of 'UserloginServiceImpl' in the main editor. The project structure includes folders 'BLL', 'DAL', 'entity', 'UL', 'utils', 'resources', 'webapp', 'test', and 'target', along with files like 'pom.xml' and 'untitled.iml'. The main editor displays the code for 'UserloginServiceImpl.java'.

```
5
6 import 三层架构.DAL.UserDao;
7 import 三层架构.DAL.UserDaoImpl;
8 import 三层架构.entity.user;
9
10 public class UserloginServiceImpl implements UserloginService {
11     //创建user用户的对象
12     private UserDao userDao = new UserDaoImpl();
13
14     @Override
15     public boolean loginUser(String name, String password) {
16         if(name == null || password == null){
17             return false;
18         }
19         user u= userDao.login(name,password);
20         if (u!=null){
21             return true;
22         }else
23             return false;
24     }
25 }
```

6 创建用户表示层view

```
<ul class="class_date">
  <li id='class_month'><a href="index.jsp"><b>全部</b></a>
    <%
      UserloginServiceImpl userService = new UserloginServiceImpl();
      userService.loginUser(name,password);
    %>
    <a href="index.jsp?tid=<%=theme.getTid()%>">
      <b><%=theme.getTname() %></b>
```

直接在前端jsp调用service方法。

另一种方式

login.jsp页面

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="UTF-8">
7 <title>Insert title here</title>
8 </head>
9 <body>
10     <form action = "doLogin.jsp" method = "post">
11         账户: <input type = "text" name = "username"/>
12         <br><br>
13         密码: <input type = "password" name = "password"/>
14         <br><br>
15         <input type = "submit" value = "登录"/>
16
17     </form>
18 </body>
19 </html>
```

将控制逻辑发送到do页面（也可以是只含业务的jsp）

doLogin.jsp页面

```
1 <%@page import="com.zking.news.biz.impl.UsersBizImpl"%>
2 <%@page import="com.zking.news.biz.IUsersBiz"%>
3 <%@page import="com.zking.news.entity.Users"%>
4 <%@ page language="java" contentType="text/html; charset=UTF-8"
5     pageEncoding="UTF-8"%>
6
7
8 <%
9     //设置request编码
10    request.setCharacterEncoding("utf-8");
11
12    //获取表单值
13    String username = request.getParameter("username");
14    String password = request.getParameter("password");
15
16
17    //实体封装
18    Users users = new Users(username,password);
19
20    //调用业务逻辑层
21    IUsersBiz iub = new UsersBizImpl();
22    Users u = iub.usersLogin(users);
23
24    if(u!=null){
25        session.setAttribute("users", u);
26        页面跳转
27    }else{
28        页面跳转
29    }
30
31 %>
```

转账页面:

```
<body>
  <form action="${pageContext.request.contextPath }/servlet/AccountServlet" method="post" >

      转出方:<input type="text" name="outaccount"/><br/>
      转入方:<input type="text" name="intaccount"/><br/>
      金额<input type="text" name="money"/><br/>
      <input type="submit" value="转账"/><br/>
  </form>
</body>
```

Servlet:

```
@WebServlet(name = "TransServlet", urlPatterns = "/trans")
public class TransServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
        request.setCharacterEncoding("utf-8");
        response.setContentType("text/html;charset=utf-8");
        // 接受数据
        String from = request.getParameter("from");
        String to = request.getParameter("to");
        String money = request.getParameter("money");
        // 验证金额
        BigDecimal m = new BigDecimal(money);
        if (m.doubleValue() <= 0) {
            response.getWriter().write("转账金额不能小于0");
            return;
        }
        // 转账
        AccountService accountService = new AccountServiceImpl();
        Account accfrom = new Account(Integer.parseInt(from), "宁宁", new BigDecimal(0));
        Account accto = new Account(Integer.parseInt(to), "春春", new BigDecimal(0));
        try {
            accountService.transMoney(accfrom, accto, m);
            response.getWriter().write("转账成功");
        } catch (Exception e) {
            e.printStackTrace();
            response.getWriter().write("转账失败");
        }
    }
}
```

项目分包:

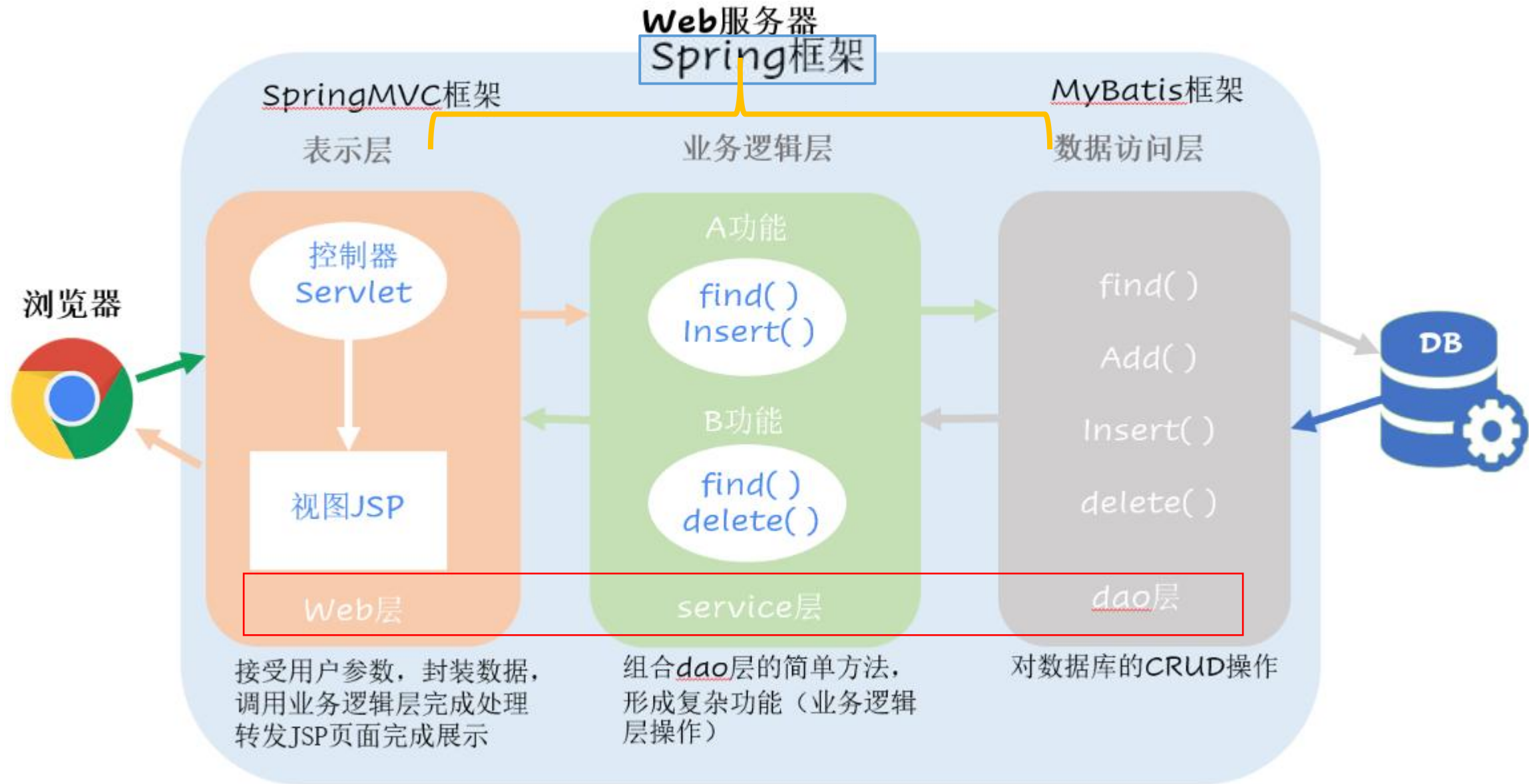
```
com.qf.domain、com.qf.dao、com.qf.dao.impl
com.qf.service、com.qf.service.impl
com.qf.utils
com.qf.web.servlet
```

- javaweb早期的实现
- 缺点:
 - 1、使用虽然简单，但是性能不好，jsp在web程序中作为一个前端的程序，不应该包含太多的业务逻辑，业务逻辑应该在后台处理，然后在前端显示处理结果，才不会给前端jsp造成压力。
 - 2、虽然开发起来更加快速，但是不便于维护，前端代码如果包含太多的业务逻辑以及显示代码，维护起来是一项繁琐的工作。
 - 3、从安全性来讲，业务逻辑直接暴露在前端，更容易让你的web程序受到攻击。瘫痪你的web程序。

SSM与三层架构的关系

SSM，即 SpringMVC、Spring 与 MyBatis 三个框架。它们在三层架构中所处的位置是不同的，即它们在三层架构中的功能各不相同，各司其职。

- SpringMVC：作为 View 层的实现者，完成用户的请求接收功能。SpringMVC 的 **Controller** 作为整个应用的控制器，完成用户请求的转发及对用户的响应。
- MyBatis：作为 Dao 层的实现者，完成对数据库的增、删、改、查功能。
- Spring：以整个应用大管家的身份出现。整个应用中所有 Bean 的生命周期行为，均由 Spring 来管理。即整个应用中所有对象的创建、初始化、销毁，及对象间关联关系的维护，均由 Spring 进行管理。



Service
Interfaces
Impls

Dao
Interfaces
Mybatis

三层架构每层之间的逻辑关系

2.5 三层优缺点

【优点】

- 开发人员可以只关注整个结构中的其中某一层；
- 可维护性高，可扩展性高可以降低层与层之间的依赖；
- 有利于标准化；
- 利于各层逻辑的复用

【缺点】

- 降低了系统的性能。如果不采用分层式结构，很多业务可以直接造访数据库，以此获取相应的数据，如今却必须通过中间层来完成
- 有时会导致级联的修改，这种修改尤其体现在自上而下的方向。如果在表示层中需要增加一个功能，为保证其设计符合分层式结构，可能需要在相应的业务逻辑层和数据访问层中都增加相应的代码
- 增加了开发成本

四 开发实践

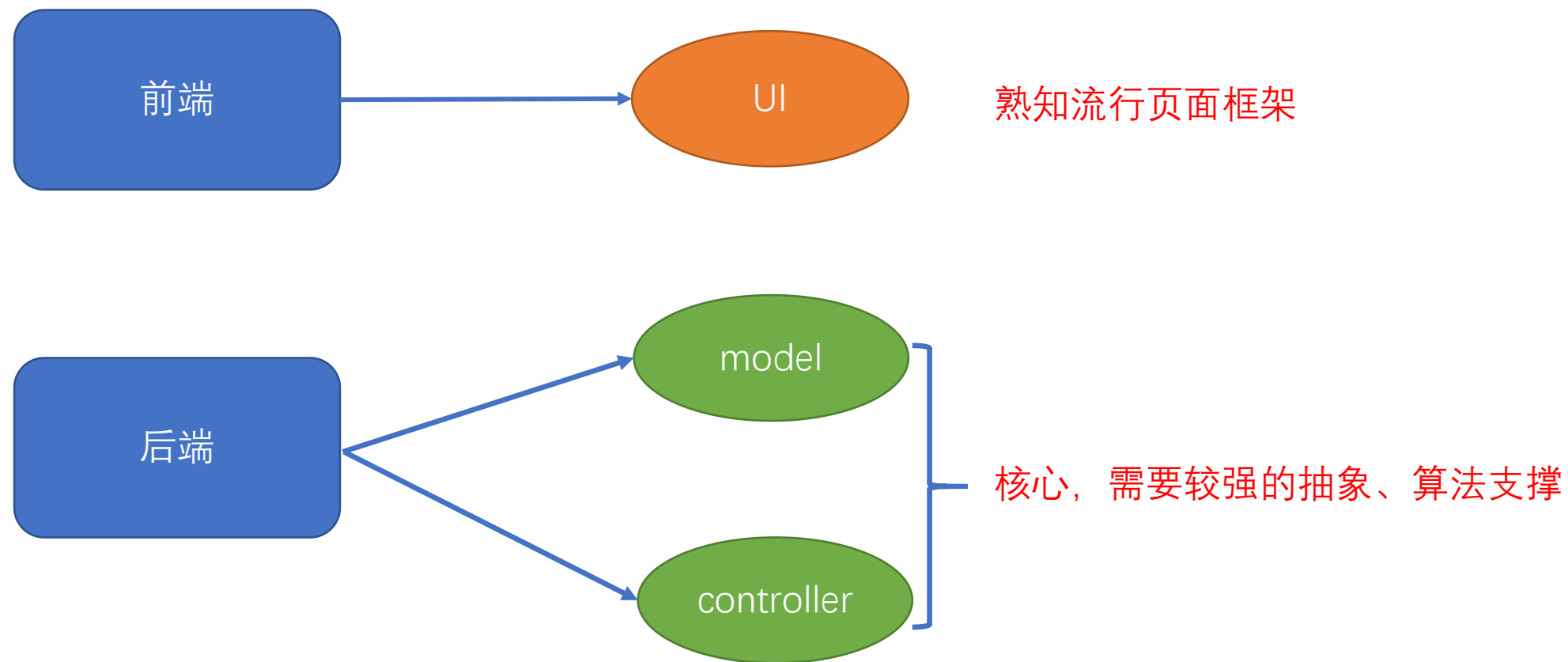
一个简单的业务流程

- 1.web视图发送一个请求 去往一个接口，比如/login
- 2.找到对应的 **Controller**，在spring 中具体路径体现为RequestMapping("/login")
- 3.访问这个路径，执行该`RequestMapping`中的逻辑
- 4.**Controller** 找到对应的**Service**
- 5.**Service**找到对应的**Dao**
- 6.**Dao**联合**Model(pojo)**操作数据库
- 7.返回结果给**Controller**
- 8.**Controller**对结果进行处理，登录成功打开主界面admin.jsp

开发技术

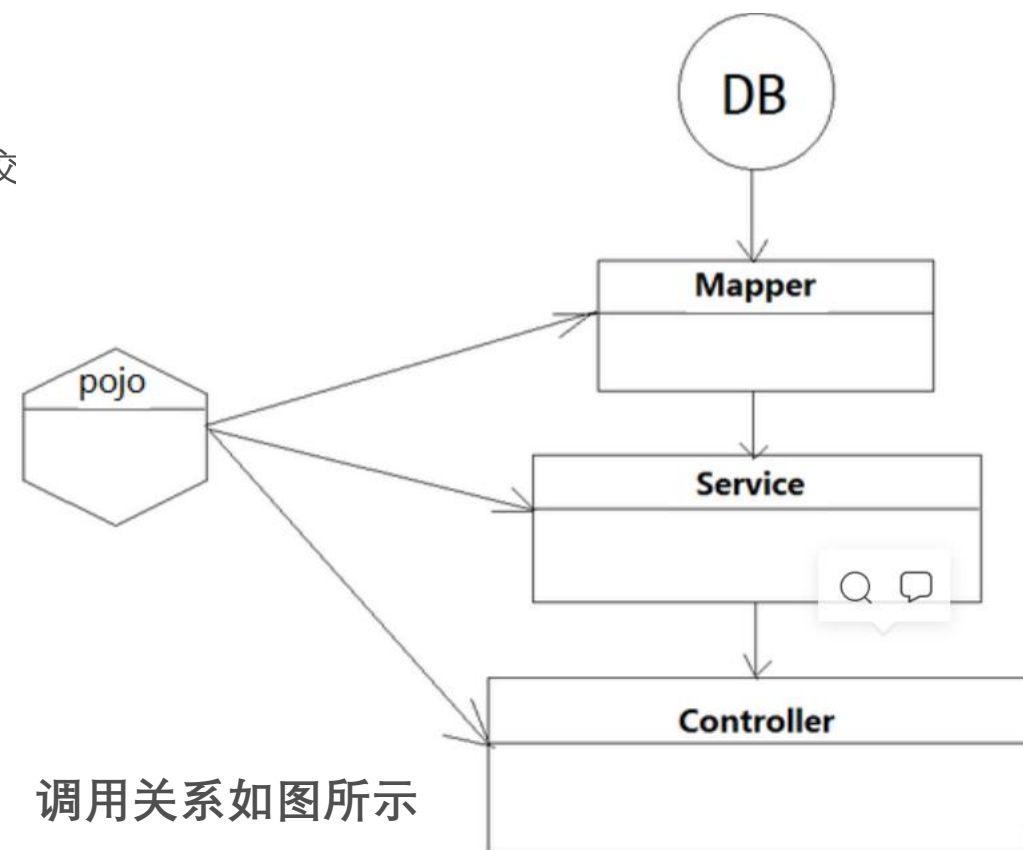
- springboot开发平台： IntelliJ IDEA
- 数据库： Mysql
- 接口测试工具： apipost、postman
- 数据库设计： powerdesigner

关键内容

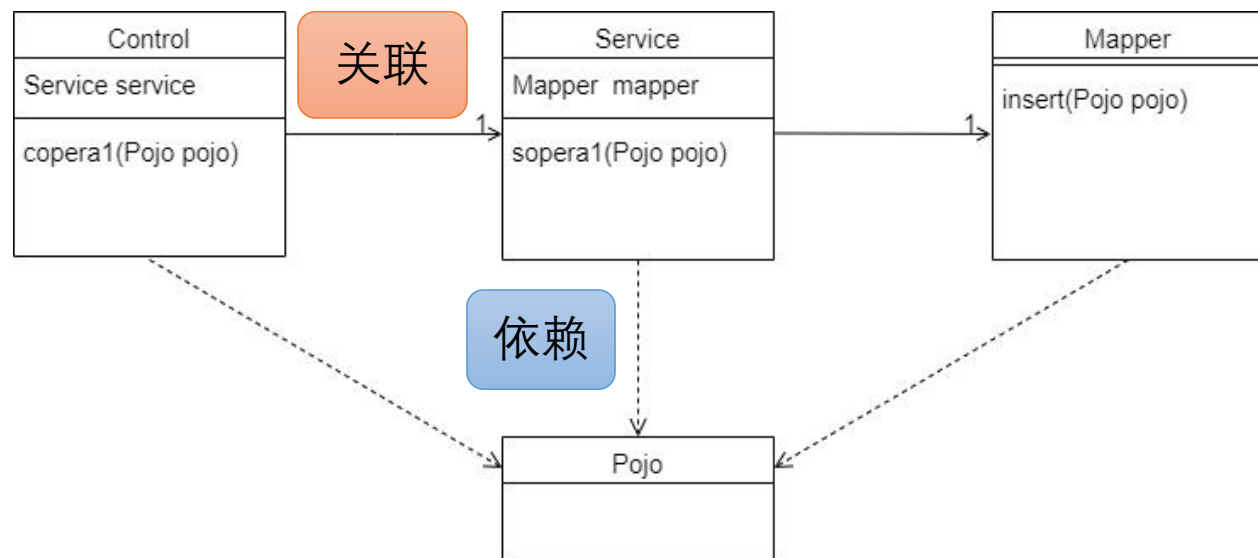


MVC的control、model在SpringBoot -- MSCM

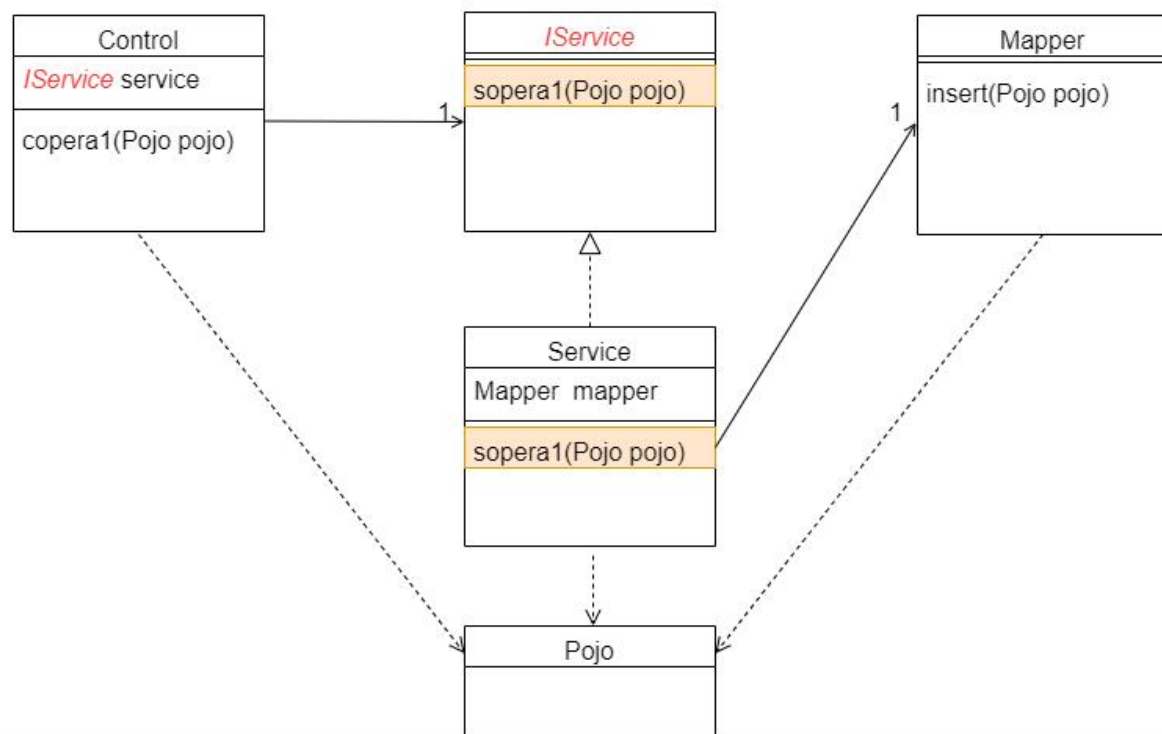
- (1) *controller*
- 控制层
- 相当于MVC的C层，*controller*通过*service*的接口来控制业务流程，也可通过接收前端传过来的参数进行业务操作。
- (2) *model (pojo)*
- 数据模型层
- 存放实体类，与数据库中的属性基本保持一致。
- (3) *service*
- 业务逻辑层
- 主要是针对具体的问题的操作，把一些数据层的操作进行组合，间接与数据库打交道
- 具体实现时先设计接口，再实现类。
- (4) *mapper (dao)*
- 数据存储对象
- *mapper*层直接与数据库打交道（执行SQL语句），接口提供给*service*层。



类图



概念



实现

- 三层架构包命名:

- 简单版

cn.itcast	公司域名倒写
cn.itcast.dao	dao 层实现
cn.itcast.service	service 层实现
cn.itcast.web.servlet	web 层实现
cn.itcast.domain	JavaBean
cn.itcast.utils	工具

- 完整版

cn.itcast	公司域名倒写
cn.itcast.xxx	项目名称
cn.itcast.xxx.yyy	子模块
cn.itcast.xxx.yyy.dao	子模块 dao 层接口
cn.itcast.xxx.yyy.dao.impl	子模块 dao 层实现类
cn.itcast.xxx.yyy.service	子模块 service 层接口
cn.itcast.xxx.yyy.service.impl	子模块 service 层实现类
cn.itcast.xxx.yyy.domain	子模块 JavaBean (子模块 yyy 可省略)
cn.itcast.xxx.yyy.web.servlet	子模块 web 层, servlet
cn.itcast.xxx.yyy.web.filter	子模块 web 层, filter
cn.itcast.xxx.utils	工具
cn.itcast.xxx.exception	自定义异常
cn.itcast.xxx.constant	常量

4.1 前端（页面）框架

- **UI框架（UI库） --基础框架：**

- Easyui (<https://www.jeasyui.net/>) --基于 jQuery 的框架,集成了各种用户界面插件
- Bootstrap (<https://www.bootcss.com/>)

- **二次封装扩展框架**

- Adminlte建立在Bootstrap3和jQuery1.11+
- H+基于 Bootstrap3.3.7 最新版本开发的扁平化主题，采用了主流的左右两栏式布局，并集成了最新的 jQuery 版本(v2.1.4)。
- 下载https://gitee.com/hplus_admin/hplus，文档https://hplus_admin.gitee.io/hplus/docs/

- **更高级企业级快速开发解决方案：**

- JeeSite 不仅仅是一个后台开发框架，它是一个企业级快速开发解决方案，后端基于经典组合 Spring Boot、Shiro、MyBatis，前端采用 Beetl、Bootstrap、AdminLTE 经典开发模式，或者分离版 Vue3、Vite、Ant Design Vue、TypeScript、Vben Admin 最先进技术栈。
- Ruoyi若依是一套全部开源的快速开发平台，毫无保留给个人及企业免费使用。使用最流行的技术 SpringBoot、SpringCloud、Shiro、Thymeleaf、Vue、Element、Bootstrap。

[首页](#) / [表单](#) / [基本表单元素](#)

圆形

占位符

辅助文本

禁用

文本

搜索框

Q 请输入关键词... X

获得焦点

文件上传

大小

大尺寸

文本域

4.2 基于easyui前端的开发

localhost:8080/admin_login.jsp

后台登录

用户名

admin

密码

123456

登录

重置

电商平台后台管理系统

欢迎您, admin

布局

tabs

tree

商品列表

商品编号

商品名称

商品类型

请选择...

商品品牌

价格:

~

查找

输入、按钮等

添加

修改

删除

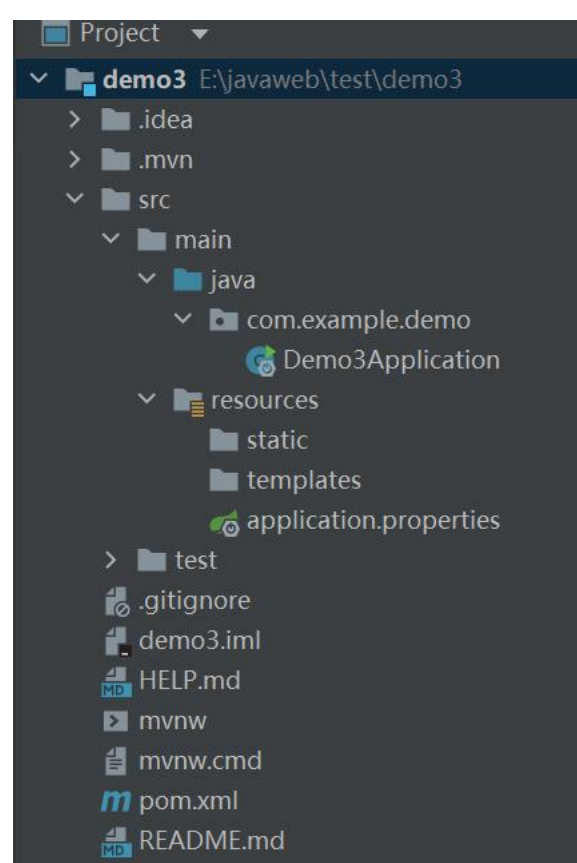
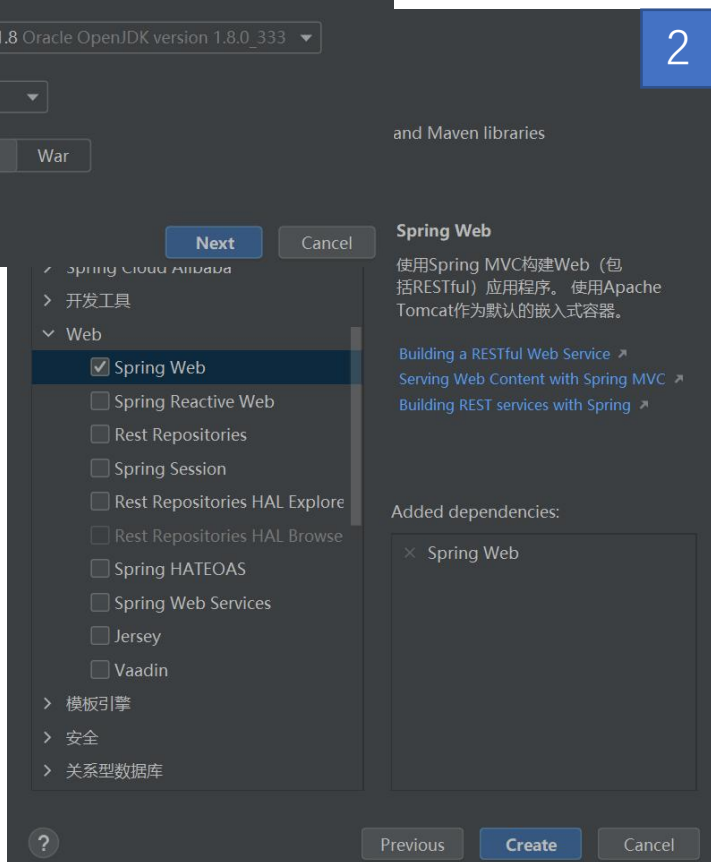
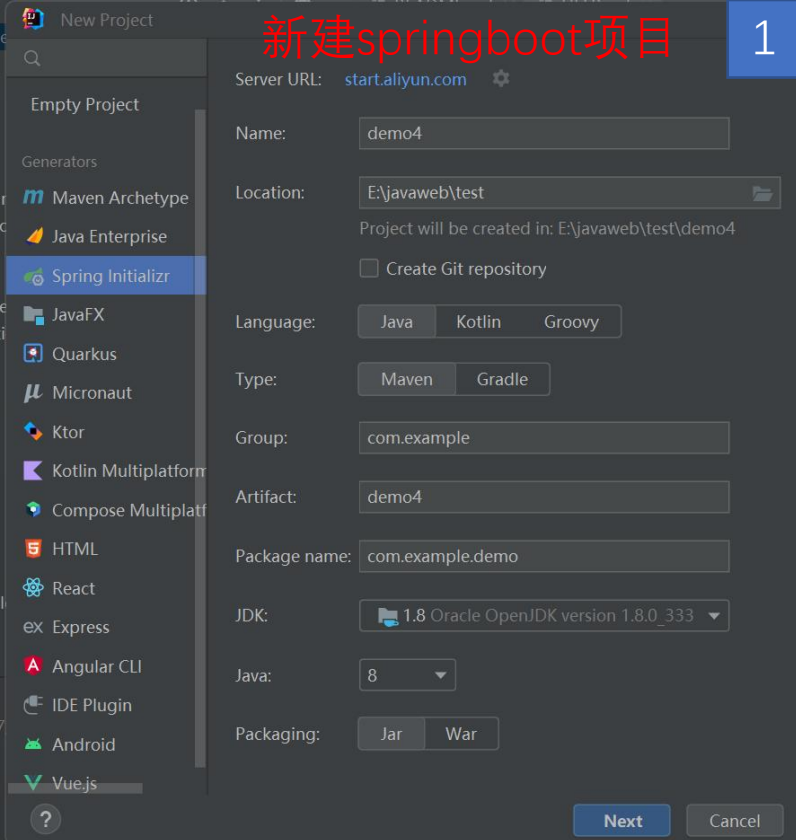
	☐	商品名称	商品类型	商品状态	商品编码	品牌	价格	库存	商品描述
1	☐	AppleMJVE2CH/A	电脑	在售	1378538	APPLE	6488	100	Apple MacBook Air 13.3英寸笔记本电脑 银色(Core i5 处理器/4GB内存/128G
2	☐	ThinkPadE450C(20EH0001CD)	电脑	在售	1309456	ThinkPad	4199	97	联想 (ThinkPad) 轻薄系列E450C(20EH0001CD)14英寸笔记本电脑(i5-4210
3	☐	联想小新300经典版	电脑	在售	1999938	联想 (Lenovo)	4399	99	联想 (Lenovo) 小新300经典版 14英寸超薄笔记本电脑 (i7-6500U 4G 500G 2
4	☐	华硕FX50JX	电脑	在售	1466274	华硕 (ASUS)	4799	100	华硕 (ASUS) 飞行堡垒FX50J 15.6英寸游戏笔记本电脑 (i5-4200H 4G 7200转
5	☐	华硕FL5800	电脑	在售	1981672	华硕 (ASUS)	4999	100	华硕 (ASUS) FL5800 15.6英寸笔记本电脑 (i7-5500U 4G 128G SSD+1TB
6	☐	联想G50-70M	电脑	在售	1904696	联想 (Lenovo)	3499	12	联想 (Lenovo) G50-70M 15.6英寸笔记本电脑 (i5-4210U 4G 500G GT820M
7	☐	美的BCD-206TM(E)	冰箱	在售	751624	美的 (Midea)	1298	100	美的(Midea) BCD-206TM(E) 206升 三门冰箱 节能保鲜 闪白银
8	☐	美的BCD-516WKM(E)	冰箱	在售	977433	美的 (Midea)	3199	100	美的(Midea) BCD-516WKM(E) 516升 对开门冰箱 风冷无霜 电脑控温 纤薄设计
9	☐	海尔BCD-216SDN	冰箱	在售	1143562	海尔 (Haier)	1699	100	海尔 (Haier) BCD-216SDN 216升 三门冰箱 电脑控温 中门 宽幅变温 大冷冻能
10	☐	海尔BCD-258WDPM	冰箱	在售	1560207	海尔 (Haier)	2699	100	海尔 (Haier) BCD-258WDPM 258升 风冷无霜三门冰箱 除菌 3D立体环绕风不

10 第 1 共2页

显示1到10,共12记录

powered by miaoyong

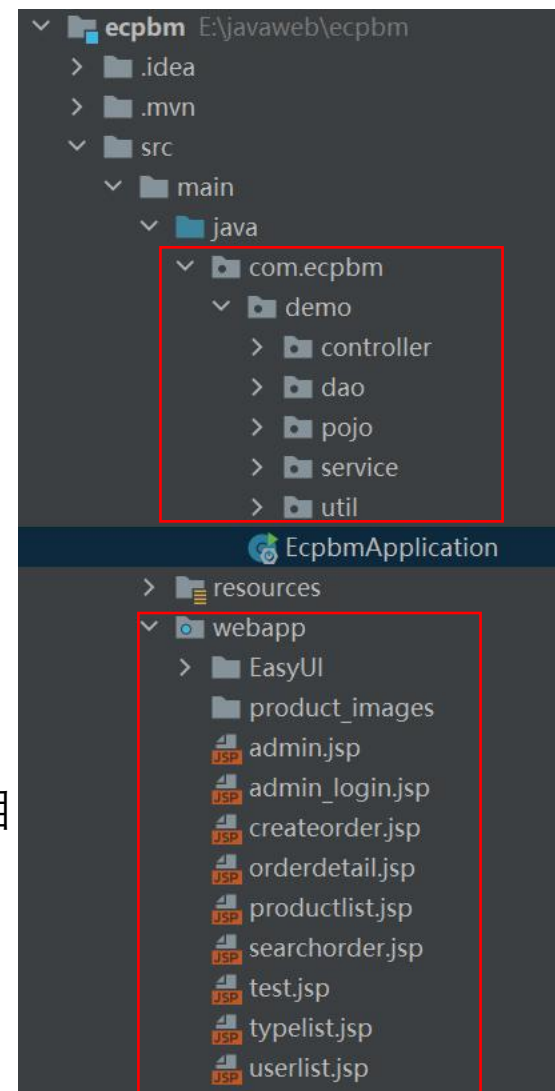
登录至主界面



添加后端



添加前端目录webapp



第一步 建立项目与目录结构

```

function checkAdminLogin() {
    $("#adminLoginForm").form("submit", {
        // 向控制器类AdminInfoController中login方法发送请求
        url : 'admininfo/login',
        success : function(result) {
            var result = eval('(' + result + ')');
            if (result.success == 'true') {
                window.location.href = 'admin.jsp';
                $("#adminLoginDlg").dialog("close");
            } else {
                $.messenger.show({title: "提示信息"...});
            }
        }
    });
}

</script>
<div id="adminLoginDlg" class="easyui-dialog"
    style="..."
    data-options="title:'后台登录',buttons:'#bb',modal:true">
    <form id="adminLoginForm" method="post">
        <table style="...">
            <tr>
                <th>用户名</th>
                <td><input class="easyui-textbox" type="text" id="name"
                    name="name" data-options="required:true" value="admin"></input></td>
            </tr>
            <tr>
                <th>密码</th>
                <td><input class="easyui-textbox" type="text" id="pwd"
                    name="pwd" data-options="required:true" value="123456"></input></td>
            </tr>
        </table>
    </form>
</div>
<div id="bb">
    <a href="javascript:void(0)" class="easyui-linkbutton"
        onclick="checkAdminLogin()">登录</a> <a href="javascript:void(0)"
        class="easyui-linkbutton" onclick="clearForm();">重置</a>
</div>

```

admin_login.jsp

```

@RequestMapping("/admininfo")
public class AdminInfoController {
    3 usages

    @Autowired
    private AdminInfoService adminInfoService;

    @RequestMapping(value = "/login", produces = "text/html;charset=UTF-8")
    @ResponseBody
    public String login(AdminInfo ai, ModelMap model) {
        // 后台登录验证
        AdminInfo admininfo = adminInfoService.login(ai);
        if (admininfo != null && admininfo.getName() != null) {
            // 验证通过后，再判断是否已为该管理员分配功能权限
            if (adminInfoService.getAdminInfoAndFunctions(admininfo.getId()).getFs().size() > 0) {
                // 验证通过且已分配功能权限，则将admininfo对象存入model中
                model.put("admin", admininfo);
                // 以JSON格式向页面发送成功信息
                return "{\"success\":\"true\",\"message\":\"登录成功\"}";
            } else {
                return "{\"success\":\"false\",\"message\":\"您没有权限，请联系超级管理员设置权限!\"}";
            }
        } else {
            return "{\"success\":\"false\",\"message\":\"登录失败\"}";
        }
    }
}

```

控制类AdminInfoController

```

<head>
<title>Insert title here</title>
<link href="EasyUI/themes/default/easyui.css" rel="stylesheet"
      type="text/css" />
<link href="EasyUI/themes/icon.css" rel="stylesheet" type="text/css" />
<link href="EasyUI/demo.css" rel="stylesheet" type="text/css" />
<script src="EasyUI/jquery.min.js" type="text/javascript"></script>
<script src="EasyUI/jquery.easyui.min.js" type="text/javascript"></script>
<script src="EasyUI/easyui-lang-zh_CN.js" type="text/javascript"></script>
</head>

```

引入资源

```

<body class="easyui-layout">
  <div data-options="region:'north',border:false"
        style="height: 60px; background: #B3DFDA; padding: 10px">
    <div align="left">
      <div style="font-family: Microsoft YaHei; font-size: 16px;">电商平台所
    </div>
    <div align="right">
      欢迎您, <font color="Red">${sessionScope.admin.name}</font>
    </div>
  </div>
  <div data-options="region:'west',split:true,title:'功能菜单'"
        style="width: 180px">
    <ul id="tt"></ul>
  </div>
  <div data-options="region:'south',border:false"
        style="height: 50px; background: #A9FACD; padding: 10px; text-align: center">powered
    by miaoyong</div>
  <div data-options="region:'center'">
    <div id="tabs" data-options="fit:true" class="easyui-tabs"
          style="width: 500px; height: 250px;"></div>
  </div>
</body>

```

tree控件

tabs控件

```

<script type="text/javascript">
// 为tree指定数据
$('#tt').tree({
  url : 'admininfo/getTree?adminid=${sessionScope.admin.id}'
});
$('#tt').tree({
  onClick : function(node) {
    if ("商品列表" == node.text) {
      if ($('#tabs').tabs('exists', '商品列表')) {
        $('#tabs').tabs('select', '商品列表');
      } else {
        $('#tabs').tabs('add', {
          title : node.text,
          href : 'productlist.jsp',
          closable : true
        });
      }
    }
  }
});

```

后台获取菜单信息

上框架

主页Admin.jsp

页面布局

第二步 建立融合easyui的前端主体html

第三步 建立增删改查CRUD页面

```
<div style="padding: 3px">
  商品编号    <input class="easyui-textbox"
    name="productinfo_search_code" id="productinfo_search_code"
    style="width: 110px" />
</div>
<div style="padding: 3px">
  商品名称    <input class="easyui-textbox"
    name="productinfo_search_name" id="productinfo_search_name"
    style="width: 110px" />         商品类型    <input
    style="width: 110px;" id="productinfo_search_tid"
    class="easyui-combobox" name="productinfo_search_tid"
    data-options="valueField:'id',textField:'name',url:'type/getType/1'"
    value="0">         商品品牌    <input
    class="easyui-textbox" name="productinfo_search_brand"
    id="productinfo_search_brand" style="width: 110px" />
  &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;价格: <input class="easyui-numberbox"
    name="productinfo_search_priceFrom"
    id="productinfo_search_priceFrom" style="width: 80px;" /> ~ <input
    class="easyui-numberbox" name="productinfo_search_priceTo"
    id="productinfo_search_priceTo" style="width: 80px;" />
  &nbsp;&nbsp;&nbsp;&nbsp;<a href="javascript:void(0)" class="easyui-linkbutton"
    iconCls="icon-search" plain="true" onclick="searchProduct();">查找</a>
</div>
```

基本控件

Productlist.jsp

```
<script type="text/javascript">
$(function() {
  $('#dg_productinfo').datagrid({
    singleSelect : false, //设置datagrid为单选
    url : 'productinfo/list', //为datagrid设置数据源
    pagination : true, //启用分页
    pageSize : 10, //设置初始每页记录数（页大小）
    pageList : [ 10, 15, 20 ], //设置可供选择的页大小
    rownumbers : true, //显示行号
    fit : true, //设置自适应
    toolbar : '#tb_productinfo', //为datagrid添加工具栏
    header : '#searchtb_productinfo', //为datagrid标头添加搜索栏
    columns : [ [ { //编辑datagrid的列
      title : '序号',
      field : 'id',
      align : 'center',
      checkbox : true
    }, {
      field : 'name',
      title : '商品名称',
      width : 200
    }, {
```

datagrid

```
@Controller
@RequestMapping("/productinfo")
public class ProductInfoController {

    7 usages
    @Autowired
    ProductInfoService productInfoService;

    // 后台商品列表分页显示
    @RequestMapping(value = "/list")
    @ResponseBody
    public Map<String, Object> list(Integer page, Integer rows, ProductInfo productInfo) {...}

    // 添加商品
    @RequestMapping(value = "/addProduct", produces = "text/html;charset=UTF-8")
    @ResponseBody
    public String addProduct(ProductInfo pi, @RequestParam(value = "file", required = false) MultipartFile file,
        HttpServletRequest request, ModelMap model) {...}

    // 修改商品
    @RequestMapping(value = "/updateProduct", produces = "text/html;charset=UTF-8")
    @ResponseBody
    public String updateProduct(ProductInfo pi, @RequestParam(value = "file", required = false) MultipartFile file,
        HttpServletRequest request, ModelMap model) {...}

    // 商品下架(删除商品)
    @RequestMapping(value = "/deleteProduct", produces = "text/html;charset=UTF-8")
    @ResponseBody
    public String deleteProduct(@RequestParam(value = "id") String id, @RequestParam(value = "flag") String flag) {...}
}
```

controller

5

```
@Mapper
public interface ProductInfoDao {

    // 分页获取商品
    1 usage
    @Results({ @Result(id = true, column = "id", property = "id"), @Result(column = "code", property =
        @Result(column = "name", property = "name"), @Result(column = "brand", property = "brand"),
        @Result(column = "pic", property = "pic"), @Result(column = "num", property = "num"),
        @Result(column = "price", property = "price"), @Result(column = "intro", property = "intro"),
        @Result(column = "status", property = "status"),
        @Result(column = "tid", property = "type", one = @One(select = "com.ecpbm.demo.dao.TypeDao.

    @SelectProvider(type = ProductInfoDynaSqlProvider.class, method = "selectWithParam")
    List<ProductInfo> selectByPage(Map<String, Object> params);

    // 根据条件查询商品总数
    @SelectProvider(type = ProductInfoDynaSqlProvider.class, method = "count")
    Integer count(Map<String, Object> params);

    // 添加商品
    1 usage
    @Insert("insert into product_info(code,name,tid,brand,pic,num,price,intro,status) "
        + "values(#{code},#{name},#{type.id},#{brand},#{pic},#{num},#{price},#{intro},#{status})")
    @Options(useGeneratedKeys = true, keyProperty = "id")
    void save(ProductInfo pi);
}
```

Dao

2

```
public interface ProductInfoService {

    // 分页显示商品
    1 usage 1 implementation
    List<ProductInfo> findProductInfo(ProductInfo productInfo, Pager pager);

    // 商品计数
    1 implementation
    Integer count(Map<String, Object> params);

    // 添加商品
    1 usage 1 implementation
    public void addProductInfo(ProductInfo pi);

    // 修改商品
    1 usage 1 implementation
    public void modifyProductInfo(ProductInfo pi);
}
```

Service接口

3

```
public class ProductInfo {

    // 商品基本信息(部分)
    2 usages
    private int type; // 商品类型
    2 usages
    private String code; // 商品编码
    2 usages
    private String name; // 商品名称
    // 关联属性
    2 usages
    private Type type;
    2 usages
    private String brand;
}
```

pojo

1

```
@Service("productInfoService")
@Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.DEFAULT)
public class ProductInfoServiceImpl implements ProductInfoService {

    8 usages
    @Autowired
    ProductInfoDao productInfoDao;

    1 usage
    @Override
    public List<ProductInfo> findProductInfo(ProductInfo productInfo, Pager pager) {...}

    @Override
    public Integer count(Map<String, Object> params) {
        return productInfoDao.count(params);
    }

    1 usage
    @Override
    public void addProductInfo(ProductInfo pi) { productInfoDao.save(pi); }

    1 usage
    @Override
}
```

Service

4

第四步 建立增删改查页面背后的model

@注解被用于应用分层,实现松耦合

4.3 开发基于H+的系统

H+ 后台主题 UI 框架

Hplus(又名 H+) 原是一款商业 UI 框架，首发于 2013 年，由 Zihan's Blog 开发并维护，属于国内早期做中后台 UI 框架的产品，虽为商业产品，但在国内拥有广泛的用户基础。由于作者自身原因，该框架已停止维护，现由官方采用 MIT 协议面向社区开源，欢迎社区贡献代码。

说明

H+是一个完全响应式，基于 Bootstrap3.3.7 最新版本开发的扁平化主题，她采用了主流的左右两栏式布局，使用了 Html5+CSS3 等现代技术，她提供了诸多的强大的可以重新组合的 UI 组件，并集成了最新的 jQuery 版本(v2.1.4)，当然，也集成了很多功能强大，用途广泛的 jQuery 插件，她可以用于所有的 Web 应用程序，如网站管理后台，网站会员中心，CMS，CRM，OA 等等，当然，您也可以对她进行深度定制，以做出更强系统。

https://gitee.com/hplus_admin/hplus



Beaut-zihan
超级管理员

- 主页
- 主页示例一
- 主页示例二
- 主页示例三
- 主页示例四
- 主页示例五

- 布局
- 统计图表
- 信箱
- 表单
- 页面
- UI元素
- 表格
- 相册
- CSS动画
- 工具



Beaut-zihan
超级管理员

- 主页
- 布局
- 统计图表
- 信箱16
- 收件箱
- 查看邮件
- 写信
- 表单
- 页面
- UI元素
- 表格
- 相册
- CSS动画
- 工具

请输入您需要查找的内容 ...

首页 布局 基本表单 表单验证 主页示例

收入月40 886,200总收入98% 订单27新订

订单

请输入您需要查找的内容 ...

首页 布局 基本表单

写信

文件夹

收件箱

发信

重要

草稿

垃圾箱

分类

工作

文档

社交

广告

客户端

标签

朋友

工作

家庭

假期

音乐

照片

表单向导 百度WebUploader 头像裁剪上传 富文本编辑器 simditor 基本表格 DataTables jqGrid 页签操作

基本 分类, 查找

每页 10 条记录 查找:

渲染引擎	浏览器	平台	引擎版本	CSS等级
Gecko	Firefox 1.0	Win 98+ / OSX.2+	1.7	A
Gecko	Firefox 1.5	Win 98+ / OSX.2+	1.8	A
Gecko	Firefox 2.0		1.8	A
Gecko	Firefox 3.0		1.9	A
Gecko	Camino 1.0		1.8	A

提示 提示信息

按钮插件 搜

带下拉框 操作

分段 操作

操作

保存内容 取消

前端布局

.navbar-static-side



.navbar-static-top

.page-heading

.wrapper-content

实现效果

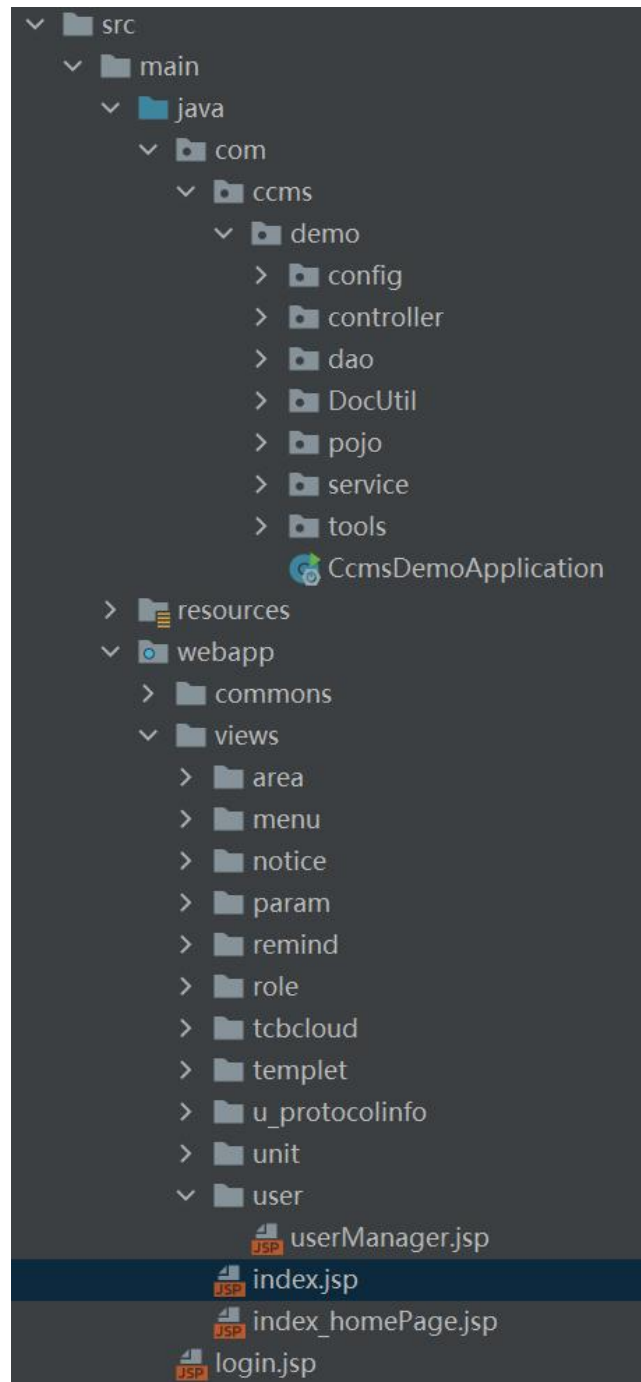


典型管理系统或网站后台界面布局

布局结构（经典布局）

在所有页面的 `<head>` 中，应包含的meta和css等如下：

```
1. <!DOCTYPE html>
2. <head>
3.
4.     <!-- Meta -->
5.     <meta charset="utf-8">
6.     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7.     <meta name="renderer" content="webkit">
8.     <title>H+ 后台主题UI框架 - 页面</title>
9.
10.    <!-- CSS文件 -->
11.    <link href="css/bootstrap.min.css" rel="stylesheet">
12.    <link href="css/font-awesome.css" rel="stylesheet">
13.    <link href="css/animate.css" rel="stylesheet">
14.    <link href="css/style.css" rel="stylesheet">
15.
16. </head>
```



- ◆在src/main/java目录下，新建一个com.ccms.demo.pojo包，在包中新建一个实体类SysUser。

```
package com.ccms.demo.pojo;  
  
public class SysUser {  
    2 usages  
    private String userCode;  
  
    2 usages  
    private String name;  
  
    2 usages  
    private String psw;  
  
    2 usages  
    private String operatorId;
```

- ◆在src/main/java目录中，新建一个com.ccms.demo.dao包，用于存放数据访问层接口。在包中新建一个接口 UserDao，在接口中声明方法。

```
2 usages
@Repository
public interface UserDao {
    // 根据登录名和密码查询合法用户
    // @Select("select * from sys_user where name=#{name} and psw=#{psw} and
    // delState=1")
    1 usage
    @SelectProvider(type = UserDynasqlProvider.class, method = "login")
    public SysUser selectByNameAndPwd(Map<String, Object> params);

    // 根据条件，分页动态查询区域管理员
    1 usage
    @SelectProvider(type = UserDynasqlProvider.class, method = "selectWithParam")
    List<SysUser> selectByPage(Map<String, Object> params);

    // 根据条件查询区域管理员总数
    @SelectProvider(type = UserDynasqlProvider.class, method = "count")
    Integer count(Map<String, Object> params);
}
```

- ◆ 在src/main/java目录中，新建一个com.ccms.demo.service包，用于存放业务逻辑层接口。在包中新建一个接口UserService，在接口中声明一个addUser方法，用于添加用户。
- ◆ 新建UserService接口的实现类UserServiceImpl，存放在com.ccms.demo.service.impl包中，以实现addUser方法。

```
package com.ccms.demo.service;

import ...

4 usages 1 implementation
public interface UserService {
    // 登录
    1 implementation
    public SysUser login(String name, String psw);

    // 添加用户
    2 usages 1 implementation
    public boolean addUser(SysUser user);
```

```
package com.ccms.demo.service.impl;

import ...

@Service("userService")
@Transactional(propagation = Propagation.REQUIRED, isolation = Isolation.DEFAULT)
public class UserServiceImpl implements UserService {

    21 usages
    @Autowired
    UserDao userDao;

    2 usages
    @Autowired
    ModuleDao moduleDao;

    2 usages
    @Override
    public boolean addUser(SysUser user) {
        Object[] params = null;
        String code = UUIDGenerator.getUUID();
        user.setUserCode(code);
        user.setDelState(1);
        int result = userDao.insertUser(user);
        return result > 0;
    }
}
```

- ◆ 在src/main/java目录中，创建包com.ccms.demo.controller，用于存放控制器类。在包中新建一个类UserController。

```
package com.ccms.demo.controller;

import ...

@Controller
@RequestMapping("/user")
public class UserController {

    21 usages

    @Autowired
    UserService userService;
```

```
// 添加用户
@RequestMapping(value = "/addUser", method = { RequestMethod.GET, RequestMethod.POST })
public String addUser(SysUser user, HttpServletRequest req, HttpServletResponse res) {
    // 从request中获取当前区域管理员的区域编号
    String areaId = req.getSession().getAttribute("AREANUMBER").toString();
    // 给对象user设置所属区域编号属性
    user.setAreaId(areaId);
    JSONObject jsonObject = new JSONObject();
    // 从request中获取当前区域管理员用户编号
    String operateId = req.getSession().getAttribute("USERID").toString();
    // 给对象user设置操作人属性
    user.setOperatorId(operateId);
    // 给对象user设置密码属性
    user.setPsw(MD5Util.MD5(user.getPsw()));
    // 判断该用户名是否存在
    String existName = userService.isExistName(user.getName(), userCode: "", areaId);
    // 判断该单位是否存在
    String existUnit = userService.isExistUnit(user.getUnitId(), userCode: "", areaId);
    if (existUnit.equals("exitUnit")) {...} else if (existName.equals("exit")) {
        jsonObject.addProperty("property: isExist", value: true);
    } else if (existName.equals("no")) {
        // 调用业务方法添加用户
        boolean result = userService.addUser(user);
        jsonObject.addProperty("property: success", result);
    }
}
```

- ◆在src/main/webapp/user目录下，新建一个登录页userManager.jsp。调用addUser

CRUD -- Usermanager.jsp

控件

触发事件

更新grid
控件

```
<!-- 添加窗口 -->
<div class="modal fade" id="addwin">
  <div class="modal-dialog" style="...">
    <div class="modal-content">
      <div class="modal-header">
        <button type="button" class="close" data-dismiss="modal"
          aria-hidden="true">×</button>
        <h4 class="modal-title" id="addwinlable">用户添加</h4>
      </div>
      <div class="modal-body">
        <div class="row">
          <form method="post" class="form-horizontal">
            <div class="form-group">
              <div class="col-sm-12">
                <input type="text" class="form-control" id="username_add">
              </div>
            </div>
            <div class="form-group">
              <div class="col-sm-12">
                <input type="password" class="form-control" id="password_add">
              </div>
            </div>
            <div class="form-group">
              <div class="col-sm-12">
                <input type="text" class="form-control" id="unitId_add">
              </div>
            </div>
            <div class="form-group">
              <div class="col-sm-12">
                <input type="text" class="form-control" id="userType_add">
              </div>
            </div>
            <div class="form-group">
              <div class="col-sm-12">
                <input type="button" value="添加" class="btn btn-primary">
              </div>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>
```

```
/**----- 添加用户-----*/
function addUser() {
  var name = $("#username_add").val();
  var psw = $("#userpsw_add").val();
  var unitId = unitId_add;
  var userType = $("#userType").val();
  console.info(unitId);
  if (userType != 1) {...}
  $.ajax({
    url : '/user/addUser',
    method : 'post',
    contentType : 'application/x-www-form-urlencoded',
    dataType : 'json',
    pagination : true, // 分页
    pageSize : 50,
    pageNumber : 1,
    //toolbar:"#tb",
    singleSelect : false,
    queryParamsType : "undefined",
    queryParams : function queryParams(params) { //设置查询参数
      var param = {
        page : params.pageNumber,
        rows : params.pageSize
      };
      return param;
    }
  });
}
```

```
var $table = $('#table');
$table
  .bootstrapTable({
    url : "/user/getUser",
    method : 'post',
    contentType : "application/x-www-form-urlencoded",
    dataType : "json",
    pagination : true, // 分页
    pageSize : 50,
    pageNumber : 1,
    //toolbar:"#tb",
    singleSelect : false,
    queryParamsType : "undefined",
    queryParams : function queryParams(params) { //设置查询参数
      var param = {
        page : params.pageNumber,
        rows : params.pageSize
      };
      return param;
    }
  });
```

```
(data.isExistUnit) {...} else if (data.success) {
  // 成功
  // 刷新
  $.ajax({
    url : '/user/refresh',
    method : 'post',
    contentType : 'application/x-www-form-urlencoded',
    dataType : 'json',
    pagination : true, // 分页
    pageSize : 50,
    pageNumber : 1,
    //toolbar:"#tb",
    singleSelect : false,
    queryParamsType : "undefined",
    queryParams : function queryParams(params) { //设置查询参数
      var param = {
        page : params.pageNumber,
        rows : params.pageSize
      };
      return param;
    }
  });
}
```

任务一

- Mysql建一张表
- 基于任一UI框架，实现表的增删改查（CRUD）
- 提交word文档介绍任务开发过程（可以详细到项目创建、页面设计、程序结构），格式不限。

RuoYi



admin

● 在线

🔒 注销

🏠 首页

⚙️ 系统管理

➕

用户管理

角色管理

菜单管理

部门管理

岗位管理

字典管理

参数设置

通知公告

日志管理

📺 系统监控

<

☰ 系统工具

<

🔗 若依官网

💻 实例演示

<

☰

文档

锁屏

全屏

若依

◀

首页

角色管理

菜单管理

部门管理

岗位管理

字典管理

▶

刷新

岗位编码:

岗位名称:

岗位状态:

所有

搜索

重置

+ 新增

✎ 修改

✖ 删除

📄 导出

🔍

🔄

📄

🗑

☐	岗位编号	岗位编码	岗位名称	显示顺序	状态	创建时间	操作
☐	1	ceo	董事长	1	正常	2022-06-07 14:57:03	✎ 编辑 ✖ 删除
☐	2	se	项目经理	2	正常	2022-06-07 14:57:03	✎ 编辑 ✖ 删除
☐	3	hr	人力资源	3	正常	2022-06-07 14:57:03	✎ 编辑 ✖ 删除
☐	4	user	普通员工	4	正常	2022-06-07 14:57:03	✎ 编辑 ✖ 删除

显示第 1 到第 4 条记录, 总共 4 条记录

© 2022 RuoYi Copyright



Beaut-zihan
超级管理员

🏠 主页

***管理



📄 文档

🔒 锁屏

🖥️ 全屏



若依



首页

角色管理



刷新

角色名称:

权限字符:

角色状态:

所有



创建时间:

开始时间

结束时间

🔍 搜索

🔄 重置

+ 新增

✎ 修改

✖ 删除

📄 导出



角色编号

角色名称



权限字符



显示顺序



角色状态

创建时间



操作



1

超级管理员

admin

1



2022-06-07 14:57:04



2

普通角色

common

2



2022-06-07 14:57:04

✎ 编辑

✖ 删除

⚙ 更多操作

显示第 1 到第 2 条记录, 总共 2 条记录

角色名

创建时

+ 新

☐

☐

☐

显示第

修改角色



* 角色名称: 普通角色

* 权限字符: common

ⓘ 控制器中定义的权限字符, 如: @RequiresRoles("")

* 显示顺序: 2

状态: ☒

备注: 普通角色

菜单权限: ☐ 展开/折叠 ☐ 全选/全不选 ☒ 父子联动

- ☒ ☒ 系统管理
- ☒ ☒ 系统监控
- ☒ ☒ 系统工具
- ☒ ☒ 若依官网

确定

关闭

Q

↺

📄

⌵

操作

✎ 编辑

✖ 删除

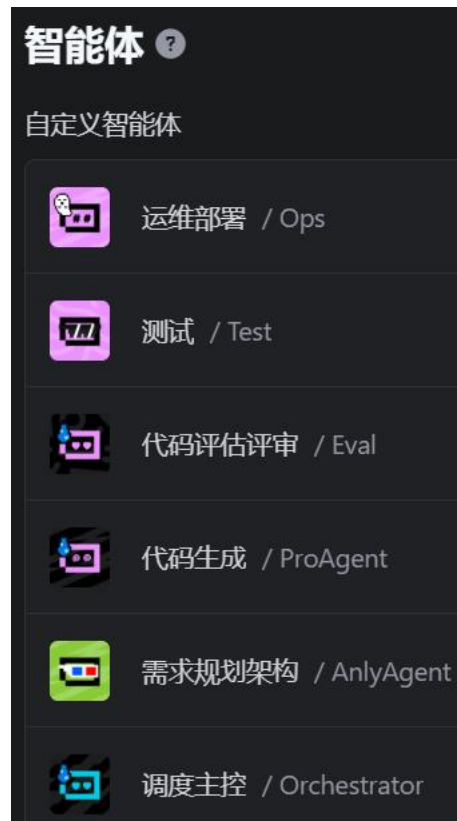
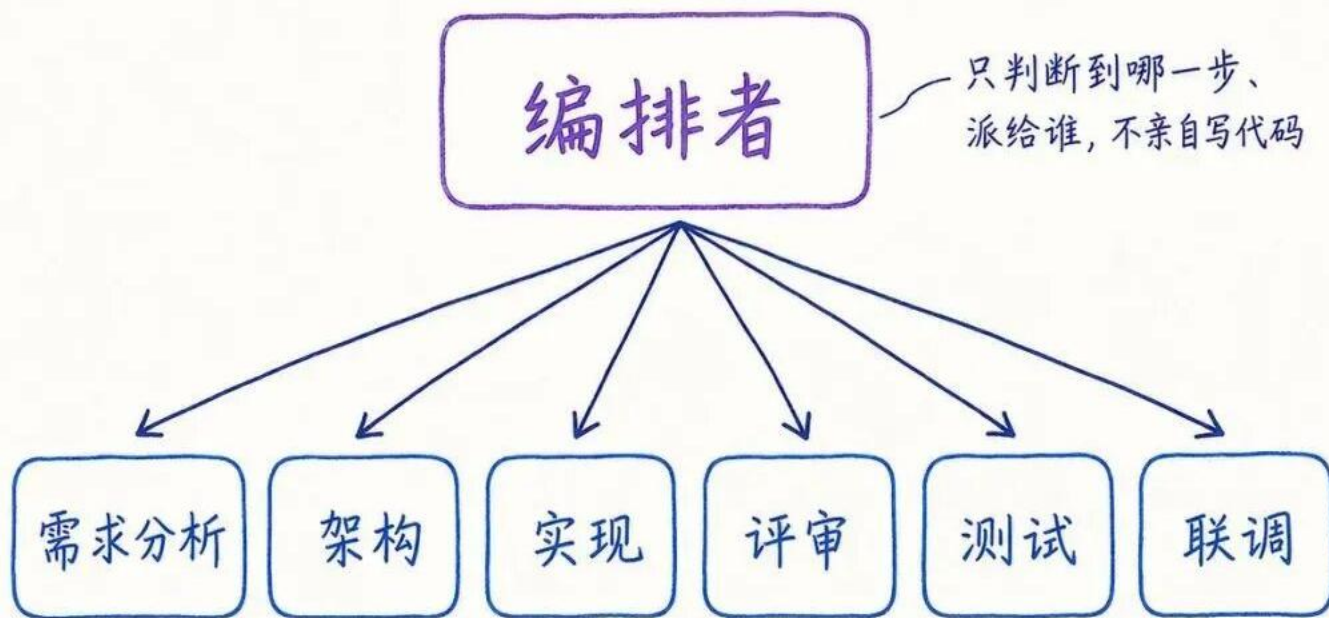
⌵ 更多操作

AI coding

- 多 Agent: 一个编排者 + 一队专职执行者

一个编排者 + 一队执行者

One orchestrator, a team of specialists



Or 调度 Agent 接收需求--定义 6 类固定分工 Agent

智能体角色	核心职责	输出产物
调度主 Agent (Orchestrator)	总控流程、分发任务、流转依赖、汇总结果、人工交互入口	PRD 总文档、任务 DAG 依赖图、进度报告
架构设计 Agent	需求拆解、数据库表设计、分层架构、接口契约、全局规范	DESIGN 架构文档、ER 图、Mermaid 架构图、API 契约
代码生成 Coder Agent	按模块生成 Entity/Mapper/Service/Controller、配置类、全局工具	分层源码、pom.xml、yml 配置、工具类
评估评审 Review Agent	代码规范校验、逻辑漏洞、SQL 注入、权限缺失、冗余代码、安全审计	评审报告、修改意见清单、合规评分
测试 Test Agent	单元测试、接口自动化测试、边界用例、异常测试、压测简易脚本	JUnit 单测、Postman 接口用例、测试报告
运维部署 Agent	数据库建表、项目打包、Jar 部署、环境配置、日志配置	SQL 初始化脚本、启动脚本、部署文档

直接生成一个可交互的单文件 HTML 原型 页面描述产品需求--prototype-html skill

聚宝盆清算系统

管理员

功能菜单

仪表盘

清算单元管理

收入核算

运营成本核算

人力成本核算

保证金核算

计提数据

报表统计

清算单元管理

管理清算单元及其关联的部门关系、渠道关系和分成关系

清算单元组管理

+ 新增清算单元

1

2

3

4

5

搜索清算单元编号/名称

全部清算单元组

查询

重置

清算单元编号	清算单元名称	清算单元组	部门关系	渠道关系	分成关系	操作
CU-2024-001	股票交易清算单元A	证券组	3	5	2	查看
CU-2024-002	基金代销清算单元B	基金组	2	4	1	查看
CU-2024-003	衍生品清算单元C	衍生品组	1	3	2	查看

共 3 条记录

上一页

1

下一页

清算单元快照

查看快照记录

1 每次进行清算计算之前，系统都会自动保存一份快照，把清算单元及其部门、渠道、分成等关系完整保存下来。如果涉及到回算，系统会使用对应时间点的快照数据进行回算，确保计算结果与当时的关系配置完全一致。

最近一次快照：2026-04-25 14:30

快照总数：12

PROTOTYPE 原型说明

以下内容将界面元素的功能说明，悬停在对应区域可高亮连线。

功能说明

1 清算单元组管理

管理清算单元组的增删改，以及组内清算单元的分配。一个清算单元只能属于一个组，确保一对一的归属关系。

2 新增清算单元

创建新的清算单元。需填写清算单元编号、名称、所属清算单元组等基本信息，创建后可继续配置部门和渠道关系。

3 清算单元列表

以表格形式展示所有清算单元，按照创建时间顺序。每行显示编号、名称、所属组，以及关联的部门关系数、渠道关系数和分成关系数。点击查看可查看详情。

4 清算单元详情

展示选中清算单元的完整信息。包含三部分一对多关系：

- 部门关系：一对多，可关联多个部门关系用于人员归属
- 渠道关系：一对多，可包含多个业务渠道关系
- 分成关系：一对多，配置与其他清算单元的收益分成比例

5 清算单元快照查看

每次进行清算计算之前，系统都会自动保存一份快照，把清算单元及其部门、渠道、分成等关系完整保存下来。如果涉及到回算，系统会使用对应时间点的快照数据进行回算，确保计算结果与当时的关系配置完全一致。

@orchestrator

项目名称：轻量化中小企业CRM客户管理系统

技术栈强制规范

后端：SpringBoot3.3 + Maven + JDK17 + MySQL8 + MyBatis-Plus + JWT + Knife4j + Lombok

规范：RESTful接口、统一返回Result、全局异常、参数校验、逻辑删除、RBAC权限

用户角色

1. 超级管理员：全部权限、账号管理、角色配置、数据导出
2. 销售员工：客户增删改查、新增跟进记录、仅查看自己名下客户

核心业务模块

1. 用户&角色权限模块

账号登录、密码加密、角色分配、接口鉴权、操作日志记录

2. 客户客户档案管理

客户新增/编辑/删除/分页多条件查询（名称、行业、来源、意向等级）

客户字段：客户名称、联系人、电话、地址、行业、客户来源、合作等级、归属销售、创建时间

支持Excel批量导入、导出客户数据；新增客户自动查重防重复

支持客户资源转移（销售离职移交客户）

3. 客户跟进记录模块

每条跟进绑定单个客户，记录沟通时间、沟通内容、跟进类型（电话/微信/面谈）、下次回访时间；

4. 客户标签管理

标签CRUD，给客户打多标签，按标签筛选客户

5. 简易数据统计

首页看板：客户总数、新增客户、待回访客户数量；按行业统计客户数量

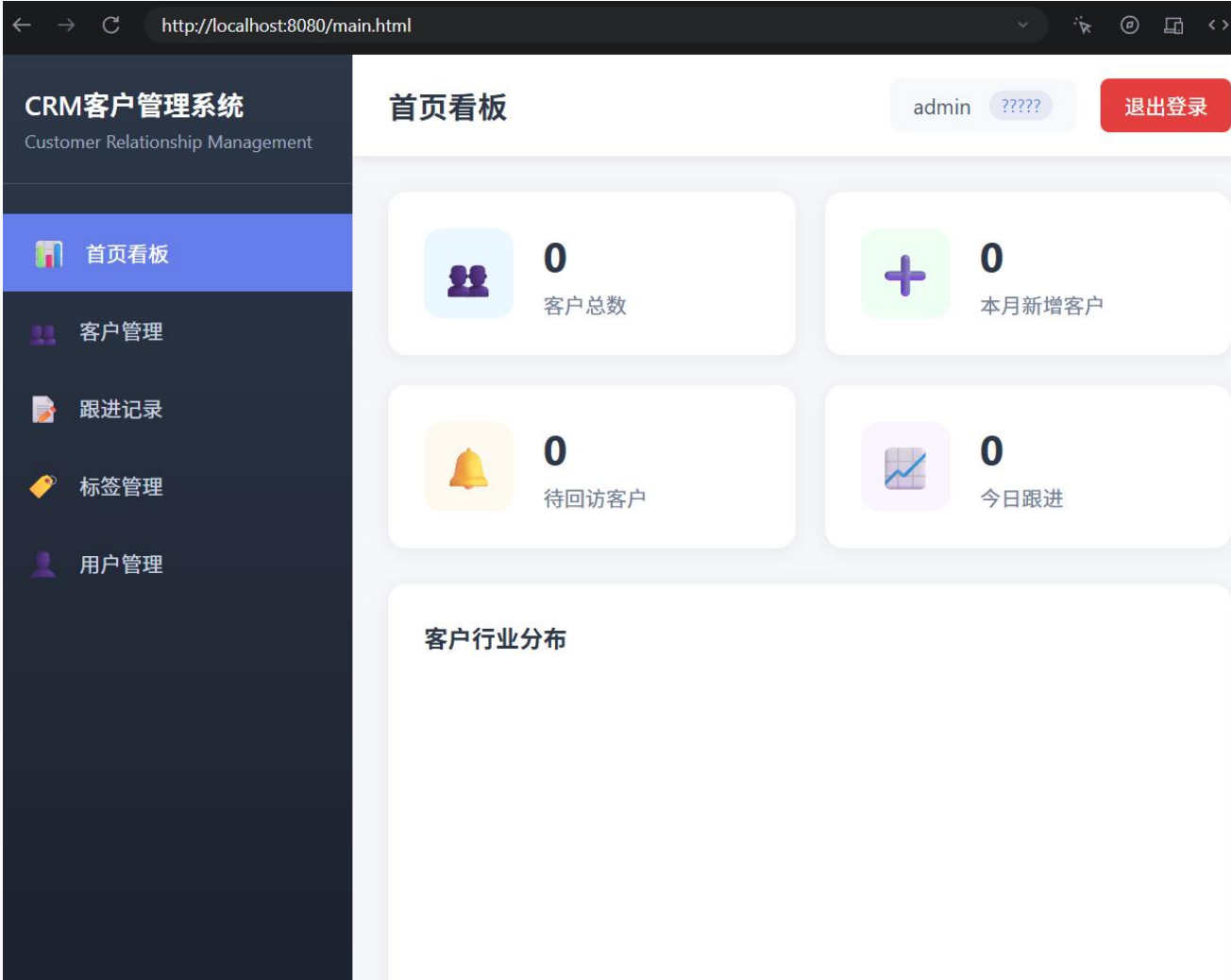
非功能要求

1. 数据库设计输出完整建表SQL，包含索引、注释、逻辑删除字段
2. 分模块并行编码，每模块完成自动调用reviewer评审代码
3. tester生成Service单元测试、接口自动化用例，全覆盖正常/异常场景
4. 全部通过后ops输出打包脚本、初始化SQL、部署文档

流水线规则

严格串行：架构设计→评审架构→分模块编码→逐模块代码评审→全量测试→部署交付
架构/代码/测试任意环节不通过，禁止进入下一阶段

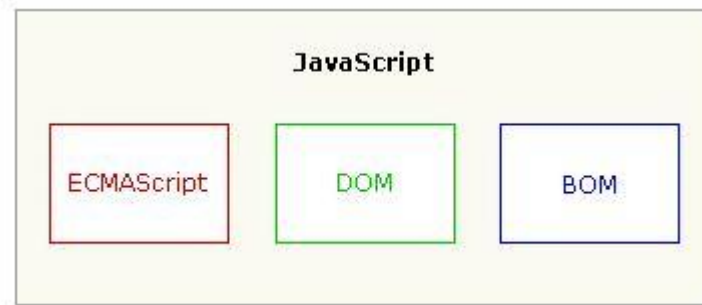
启动全套多Agent流水线开发轻量化CRM系统



附录

JavaScript基础

- JavaScript添加页面动态效果，提供用户操作体验。主要应用有：
嵌入动态文本于HTML页面、对浏览器事件作出响应、读写HTML元素、验证提交数据、检测访客的浏览器信息等。



ECMAScript核心，描述了语言的语法、变量、关键字、保留字、数据类型、语句、函数、和基本对象等等

DOM 浏览器对象模型，描述了处理网页内容的方法和接口，包含整个HTML页面的内容

BOM 文档对象模型，操作文档中的元素和内容，描述了与浏览器进行交互的方法和接口，包含整个浏览器的内容。

jQuery

- 辅助 JavaScript 开发的 js 类库
- 简便的JavaScript设计模式，优化HTML文档操作、选择 DOM 元素、事件处理、动画设计和Ajax交互

\$ 是 jQuery 的核心函数，能完成 jQuery 的很多功能。\$()表示调用\$这个函数

- `$()`传入参数为函数时：表示页面加载完成之后执行函数。相当于 `window.onload = function(){}`
- `$()`传入参数为HTML 字符串时：会为我们创建这个 HTML 标签对象，例如 `$("<div><span>div-span</span></div>")`，比原生JavaScript创建HTML标签对象的方式要简便。
- `$()`传入参数为选择器字符串时：
- `$("#id 属性值");` id 选择器，根据 id 查询标签对象，相当于`document.getElementById()`
- `$("标签名");` 标签名选择器，根据指定的标签名查询标签对象，相当于`document.getElementsByTagName()`
- `$(".class 属性值");` 类型选择器，可以根据 class 属性查询标签对象
- `$()`传入参数为DOM 对象时：会把这个 dom 对象转换为 jQuery 对象

springboot不支持jsp

- 要访问jsp，在pom.xml文件导入jsp，servlet，web相关依赖：

```
<!--添加jsp引擎，SpringBoot内置的Tomcat中没有此依赖-->  
<dependency>   
  <groupId>org.apache.tomcat.embed</groupId>   
  <artifactId>tomcat-embed-jasper</artifactId>   
</dependency> <!--jstl-->   
<dependency>   
  <groupId>javax.servlet</groupId>   
  <artifactId>jstl</artifactId>   
</dependency>
```

前端form表单参数透过@RequestMapping传递给后台Java Bean ([POJO](#))
(已录相关视频 - *easyui*中datagrid获取后台数据+前端绑定数据+后端如何获取前端数据)

```
<body>
  <div class="container">
    <form class="form-signin" action="./login" method="post">
      <h2 class="form-signin-heading">请 登 录</h2>
      <input type="text" class="form-control" placeholder="账号" name="username"/>
      <input type="password" class="form-control" placeholder="密码" name="password"/>
      <button class="btn btn-lg btn-primary btn-block" type="submit">登录</button>
    </form>
  </div>
</body>
```

前端

```
public class UserVO {
    public String username;
    public String password;
}
```

```
@Controller
public class LoginController {

    @RequestMapping(value = "login", method = RequestMethod.POST)
    public String login(UserVO user){
        System.out.println("POJO: " + user.getClass().getName() +
            ", hash code: " + user.hashCode() + ", " + user.toString());
        return "redirect:/";
    }
}
```

Rest风格

Rest，表先形式状态转化
传统风格资源描述形式：

- http://localhost/user/getById?id=1

而Rest风格描述形式：

- http://localhost/user/1

显而易见，Rest风格有以下优势：

- 隐藏资源的访问行为，无法通过地址得知对资源是哪种操作
- 简化书写

Rest风格访问资源时使用行为动作区分对资源进行了哪种操作

url	作用	请求方式
http://localhost/user	查询全部用户的信息	GET（查询）
http://localhost/user/1	查询指定用户的信息	GET（查询）
http://localhost/users	添加用户信息	POST（新增/保存）
http://localhost/users	修改用户信息	PUT（修改/更新）
http://localhost/user/1	删除用户信息	DELETE（删除）

以上的行为均为约定方式
所谓约定方式就是大家说好了的，不是规范可以打破，所以称为REST风格，而不是REST规范
描述模块的名称通常使用复数，也就是加s的格式描述，表示该类资源，而并非单个资源，例如users、books...
根据Rest风格对资源的访问称为Restful

```

@Controller
public class bookController {
    @RequestMapping("/save")
    @ResponseBody
    public String save(){
        System.out.println("user save....");
        return "{\"module':'user save'}";
    }
    @RequestMapping("/delete")
    @ResponseBody
    public String delete(int id){
        System.out.println("user delete...." + id);
        return "{\"module':'user delete'}";
    }
    @RequestMapping("/update")
    @ResponseBody
    public String update(@RequestBody User user){
        System.out.println("user update...." + user);
        return "{\"module':'user update'}";
    }
    @RequestMapping("/getById")
    @ResponseBody
    public String getById(int id){
        System.out.println("user getById...." + id);
        return "{\"module':'user getById'}";
    }
}

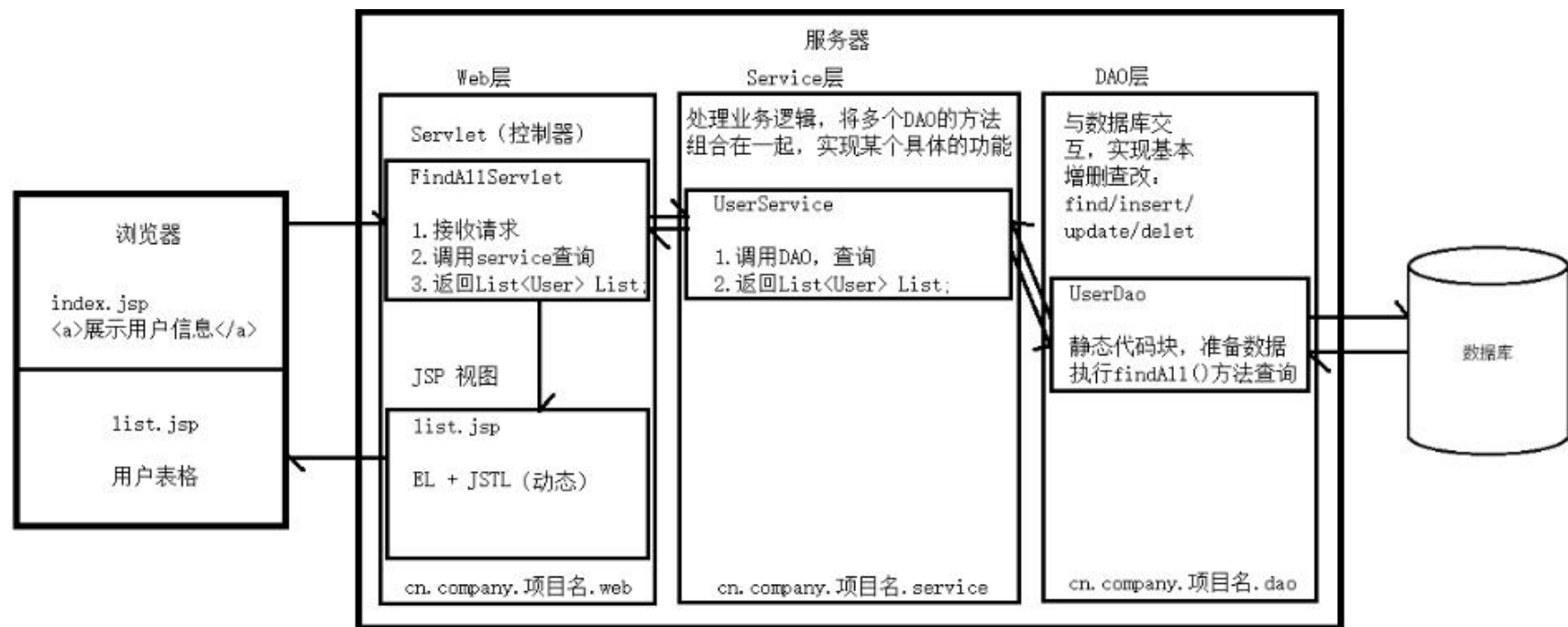
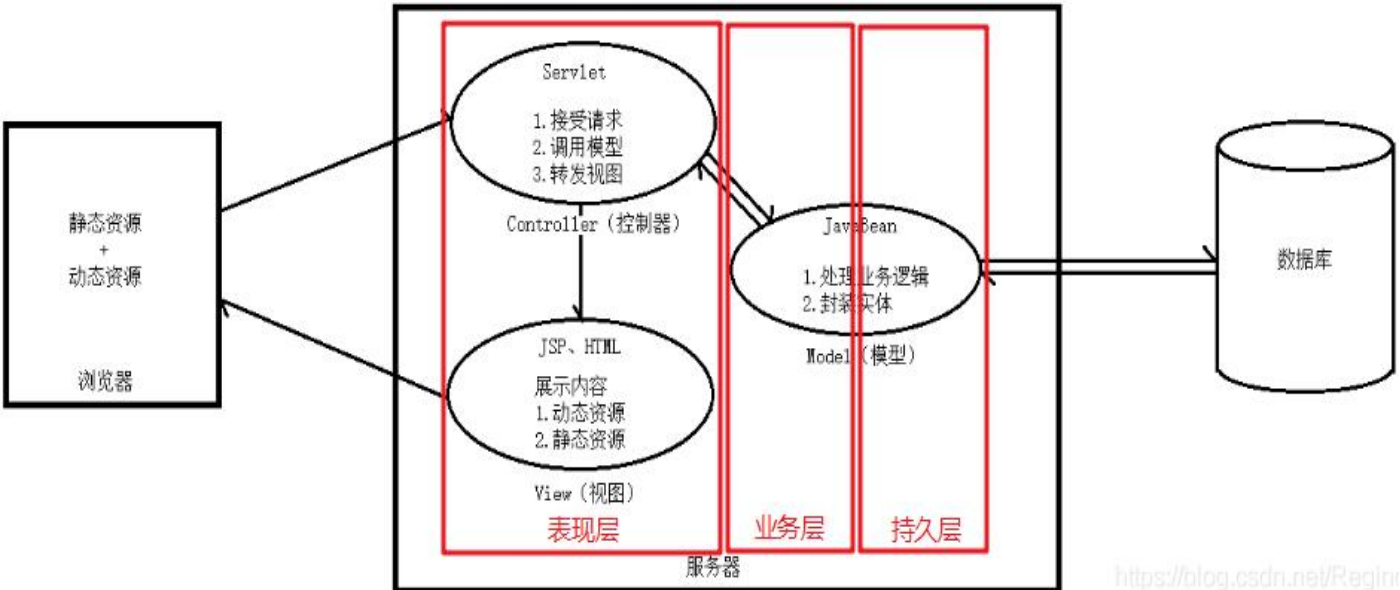
```

代码改装为
*Rest*风格

```

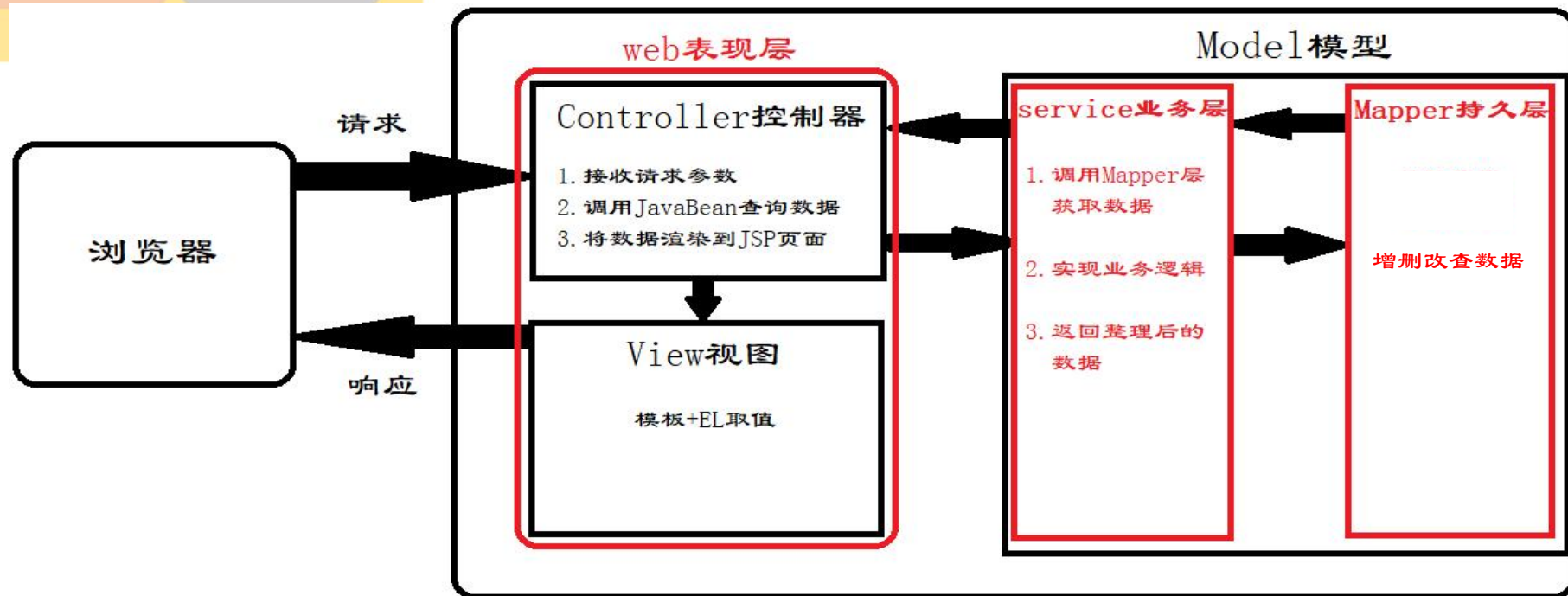
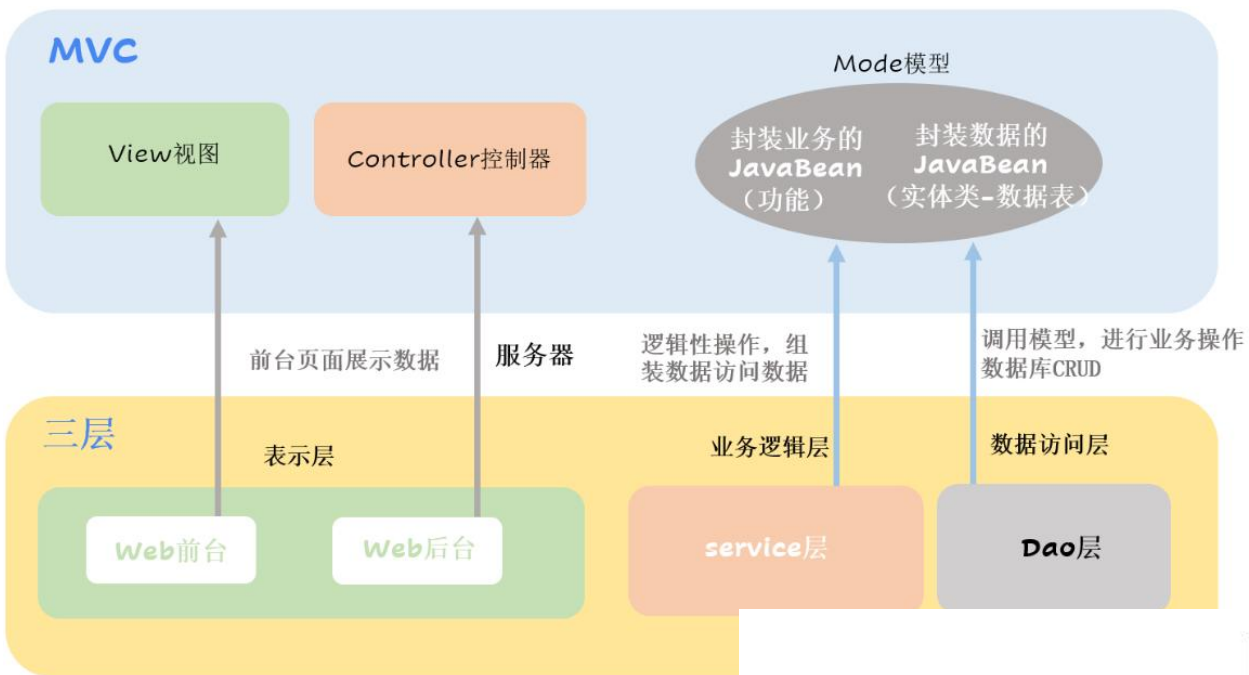
// @Controller
// @ResponseBody
@RestController
@RequestMapping("/books")
public class bookController {
    @PostMapping
    public String save(){
        System.out.println("user save....");
        return "{\"module':'user save'}";
    }
    // @RequestMapping("/{id}", method = RequestMethod.DELETE)
    @DeleteMapping("/{id}")
    public String delete(@PathVariable int id){
        System.out.println("user delete...." + id);
        return "{\"module':'user delete'}";
    }
    // @RequestMapping( method = RequestMethod.PUT)
    @PutMapping
    public String update(@RequestBody User user){
        System.out.println("user update...." + user);
        return "{\"module':'user update'}";
    }
    @GetMapping("/{id}")
    public String getById(@PathVariable int id){
        System.out.println("user getById...." + id);
        return "{\"module':'user getById'}";
    }
}

```



各层间会传递一些实体对象:
cn. company. domain (pojo/vo)

MVC与三层架构的关系



easyui跟vue

- easyui是一款UI框架，Vue是一款流行的JavaScript框架，它们都有自己的优点和使用场景。
- 简单的Vue组件，其中包含了easyui的datagrid组件

Vue与Element-UI

- Vue.js是一套用于构建用户界面的**渐进式框架**。免除JavaScript中的dom操作，简化书写，实现数据双向绑定。
- **渐进式**是指，它既可以作为一个库使用，又可以作为一个框架使用，两个之间自如选择。库是不具有侵略性，在原有项目可以引入作为第三方依赖使用；框架是具有侵入性的，整个项目是用vue构建的。

前端技术栈发展



前端技术栈的发展主要经历了三大阶段：

第一阶段，基础静态页面阶段。用的是HTML、CSS、JavaScript原生技术。

第二阶段，是前端库/封装库的出现，如Query。他们解决了浏览器原生API效率低和兼容性的问题。实质上，他们是把原生的API做了二次封装，操作dom更加简明，便于开发和掌握。

第三阶段，也就是当今时代，可以称为“组件化的时代”。这主要因为node的出现，前端开发者可以借助node开发各种自动化工具，webpack/各种包管理器，进入催生了一批前端框架的诞生，如Vue.js/React/Angular。前端开发逐步工具化、工程化。

打包编译都可以用其他语言实现，但是Node.js出现后，js从只能跑在浏览器上转变为能跑在操作系统上，熟悉js语言的前端工程师更容易开发出一系列自动化工具，前端生态繁荣以来。

vue.js等框架依赖许多底层的提供能力，它定义的语法规则需要解析，就依赖于语法解析Bable、webpack打包模块等。

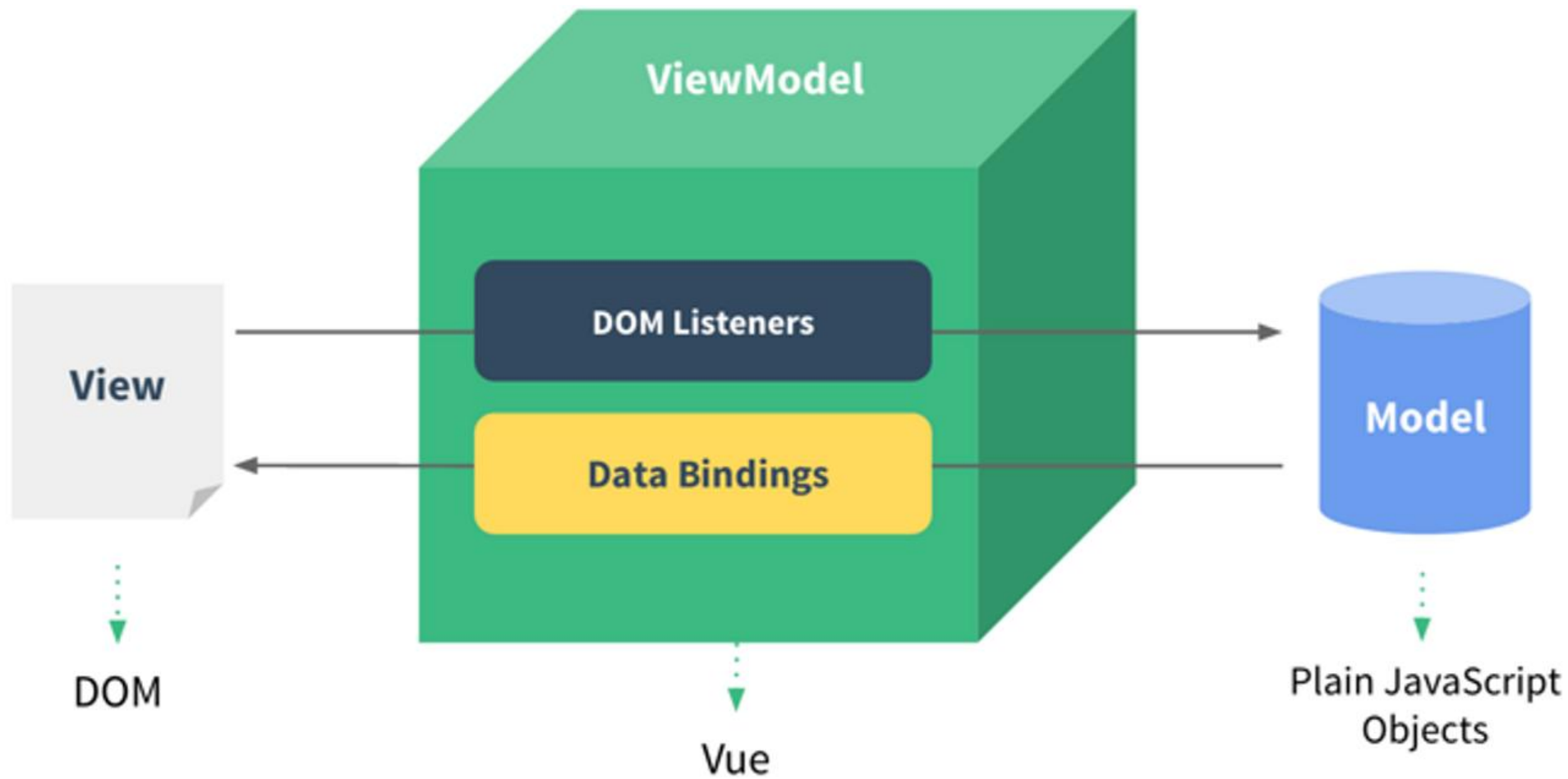
Vue.js三大核心

- Vue.js的核心是“**数据驱动视图**”
- 它有三大核心特征， 响应式/数据驱动、组件化、虚拟DOM。

MVVM模型

- 软件架构模式。
- 它包括View、Model和ViewModel三个部分。通俗来说，View是视图；Model是模型，指存储数据的地方；ViewModel和MVVM的核心，是一个调度者，负责控制数据在view 和model之间的流动。
- MVVM模式的主要目的是为了分离(View)和(Model)，二者通过ViewModel交互。如此一来，具有低耦合和可重用性的特点。
- **低耦合性**：视图可以独立于Model变化和修改，一个ViewModel可以绑定到不同的"View"上。
- **可重用性**：在ViewModel中放入视图逻辑，多个View即可重用这部分的视图逻辑。

vue实现MVVM模型



View是用户界面的DOM结构、Model 是JS简单对象，存数据的地方。ViewModel是Vue的实例，由它实现了数据的双向绑定。

Vue()是一个构造函数，用new Vue(构造参数)得到的就是一个实例，Vue实例对象是Vue.js中最基本的单元。实例参数选项有el、data、template、components等。在项目中，在main.js中通过创建最外层的Vue实例对象来实现根结点、根组件的功能。

图中ViewModel,也就是vue实例，包含DOM Listeners和Data Bindings两个工具。Data Bindings 用于将数据绑定到 视图显示，DOM Listeners 用于监听操作。

从Model侧向左看，更新Model中的数据时，Data Bindings工具会更新页面中的DOM元素，这是**数据的单向绑定**；从View侧向右看，DOM Listeners工具会监测页面上DOM元素的变化，如果有变化，则更改Model中的数据，如此，完成数据的**双向绑定**，也就实现了**数据驱动视图**。

Vue可以实现数据双向绑定的原理是：（当一个普通 JavaScript 对象传给 Vue 实例来作为它的 data 选项时）vue 首先劫持数据遍历他们的属性，配合发布者-订阅者模式的方式，通过js原生api, Object.defineProperty() 来劫持各个属性的 setter 和 getter，在数据变动时发布消息给订阅者，触发（订阅者）相应监听回调。在此基础上，vue还监听了元素的onChange(), 通知更改model层里的数据。实现了双向绑定。

Spring Boot 整合MyBatis （注解版与XML版 比较）

☀️☀️ 2.1、查询

注解:

```
1 @Select("select * from user where id = #{id}")
2 User getUserById(int id);
```

XML:

```
1 <resultMap id="userMap" type="com.dong.mybatis.model.User">
2   <id property="id" column="id"/>
3   <result property="username" column="username"/>
4   <result property="password" column="password"/>
5 </resultMap>
6 <select id="getUserById" resultMap="userMap">
7   select * from user where id = #{id};
8 </select>
```

☀️☀️ 2.2、插入

注解:

```
1 @Insert("insert into user (username,password)values(#{username},#{password})")
2   @SelectKey(statement = "select last_insert_id()",keyProperty = "id",before = false,resultType = Integer.class)
3   int addUser(User user);
```

XML:

```
1 <insert id="addUser" parameterType="com.dong.mybatis.model.User" useGeneratedKeys="true" keyProperty="id">
2   insert into user (username,password)values(#{username},#{password});
3 </insert>
```

☀️☀️ 2.3、删除

注解:

```
1 @Delete("delete from user where id = #{id}")
2   int deleteById(int id);
```

XML:

```
1 <delete id="deleteById">
2   delete from user where id = #{id};
3 </delete>
```

☀️☀️ 2.4、更新

注解:

```
1 @Update("update user set username = #{username} where id=#{id}")
2   int updateUser(int id,String username);
```

XML:

```
1 <update id="updateUser">
2   update user set username = #{username} where id=#{id};
3 </update>
```


动态SQL注解@SELECTPROVIDER、
@INSERTPROVIDER、@UPDATEPROVIDER、
@DELETEPROVIDER

MyBatis 3.x 版本提供了以下4个CRUD的高级注解。

@SelectProvider：用于构建动态查询SQL。

@InsertProvider：用于构建动态新增SQL。

@UpdateProvider：用于构建动态更新SQL。

@DeleteProvider：用于构建动态删除SQL。

动态SQL注解主要用于编写动态SQL。这里以@SelectProvider为例，它主要包含两个注解属性，其中，type表示工具类，method表示工具类的某个方法（用于返回具体的SQL语句）。

以下代码可以构建动态SQL，实现查询功能：

```
1  /**
2   * 根据用户ID，获取用户信息
3   * @author pan_junbiao
4   */
5  @SelectProvider(type = UserSqlBuilder.class, method = "buildGetUserByIdSql")
6  public UserInfo getUserById(@Param("userId") int userId);
```

UserSqlBuilder工具类的代码如下：

```
1  public class UserSqlBuilder
2  {
3      public String buildGetUserByIdSql(@Param("userId") int userId)
4      {
5          return new SQL()
6          {
7              {
8                  SELECT("*");
9                  FROM("tb_user");
10                 WHERE("user_id = #{userId}");
11             }
12         }.toString();
13     }
14 }
```

@SelectProvider, @Select和xml用法对比

```
public interface AdmainMapper{

1  @Select("SELECT * FROM userinfo WHERE username = #{username} AND password = #{password}")
2      @Results(value = {      //@Result这项可以不写
3          @Result(id = true, column = "id", property = "id"),
4          @Result(column = "username", property = "username"),
5          @Result(column = "password", property = "password") })
6      public Admin selectAdmin(@Param("username") String username,
7                               @Param("password") String password);//参数可以是一个对象, sql语句会自动匹配
8  }
```

使用@Select注解的方式很简单但是不是很灵活，对于动态条件查询是无法实现的，这时，我们可以使用xml注解的方式。

@Select注解的方式和xml的方法selectAdmin和getAdminByConditions写在同一个类中。

```
public interface AdminMapper {
    public List<Admin> getAdminByConditions(@Param("username")String username,
                                           @Param("password")String password,
                                           @Param("start")int start,
                                           @Param("limit")int limit);//参数可以直接传入一个类，在xml中可以自动匹配
}
```

xml文件

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-
mapper.dtd">
<!-- 上面类的路径-->
<mapper namespace="com.xxx. ... .AdminMapper">
<select id="getAdminByConditions" parameterType="com. ... .StudentVo" resultMap="studentResultMap">
  <![CDATA[ SELECT ]]>    //此id要和mapper中的方法名对应
  <include refid="Base_Column_List" />
  <![CDATA[ FROM student WHERE 1=1]]>
  <if test="id != null"><![CDATA[ AND id = #{id} ]]></if>
  <if test="name != null"><![CDATA[ AND name like CONCAT('%',{name},'%') ]]></if>
  <if test="age != null"><![CDATA[ AND age = #{age} ]]></if>
  <![CDATA[ ORDER BY id DESC limit #{startRow} , #{pageSize} ]]>
</select>
```

@SelectProvider方式

这种方式集成了前两种方法的优点，sql写法更符合日常开发习惯（拼接写sql），用法如下，AdmainMapper里面再加一个方法

```
public interface AdmainMapper{
@SelectProvider(type = SqlBuilder.class, method = "queryList")//指定所用sql语句@ResultMap("student")
List<student> getList(@Param("start") int start,@Param("pageSize") int pageSize, @Param("aplds") String aplds, @Param("model") String model);
}

public class SqlBuilder {//与type对应
    public String queryList(@Param("start") int start, @Param("pageSize") int pageSize, //于method对应
        @Param("aplds") String aplds, @Param("model") String model) {

        String sql = " SELECT ap.* FROM ap WHERE ap.id NOT in ( SELECT apid FROM ap_activity_mapping WHERE del = 0 ";
        if (StringUtils.isEmpty(aplds)) {
            sql += " and apid not in (" + aplds + ") ";
        }
        sql += " ) ";
        if (StringUtils.isEmpty(model)) {
            sql += " and model = " + model + "";
        }
        sql += " GROUP BY ap.id limit " + start + "," + pageSize;
        return sql;
    }
}
```

总结下来，@select注解与@SelectProvider组合起来使用，简单sql直接用@select，复杂用SelectProvider动态注解